Telephoto

Készítették: Barnai Benjámin, Farkas Tamás, Hegedűs Kevin

Tartalomjegyzék

1 Bevezetés ... 4
2 Fejlesztői dokumentáció ... 5
2.1 Adatbázis ... 5
2.1.1 Technikai környezet ... 5
2.1.2 Kapcsolatok ... 6
2.1.3 Táblastruktúrák ... 6
2.1.4 Eseményindítók ... 10
2.2 Kiemelendő Algoritmusok ... 11
2.2.1 Backend ... 11
2.2.2 Frontend ... 13
2.3 Tesztelés ... 20
2.3.1 Bevezetés ... 20
2.3.2 Technikai környezet ... 20
2.3.3 Tesztesetek ... 21
2.3.4 Teszt lefedettség ... 31
2.3.5 Teszt futtatása ... 31
2.4 Továbbfejlesztési Lehetőségek ... 32
2.4.1 Élő Adás ... 32
2.4.2 Videohívás ... 32
2.4.3 Eltűnő Posztok ... 32
2.4.4 Fotós Piactér ... 33
3 Felhasználói dokumentáció ... 34
3.1 A program célja ... 34
3.2 A program funkciói ... 34
3.2.1 Regisztráció és Belépés ... 34
3.2.2 Főoldal ... 34

3.2.3 Posztolás ... 34
3.2.4 Térkép ... 34
3.2.5 Profil ... 35
3.2.6 Admin ... 35
3.3 Szükséges hardver és szoftvereszközök ... 35
3.3.1 Hardver ... 35
3.3.2 Szoftver ... 35
3.4 Telepítés és/vagy indítás részletei ... 35
3.5 A program részletes bemutatója ... 36
3.5.1 Bejelentkezés és Regisztráció ... 36
3.5.2 Navigációs sáv ... 37
3.5.3 Főoldal ... 37
3.5.4 Térkép ... 43
3.5.5 Profil ... 44
3.5.6 Admin ... 48
4 Csapatmunka megvalósítása ... 54
5 Összefoglalás ... 55
6 Ábrajegyzék ... 56
6.1 Képek ... 56
6.2 Táblázatok ... 57
7 Irodalomjegyzék ... 59

1 Bevezetés

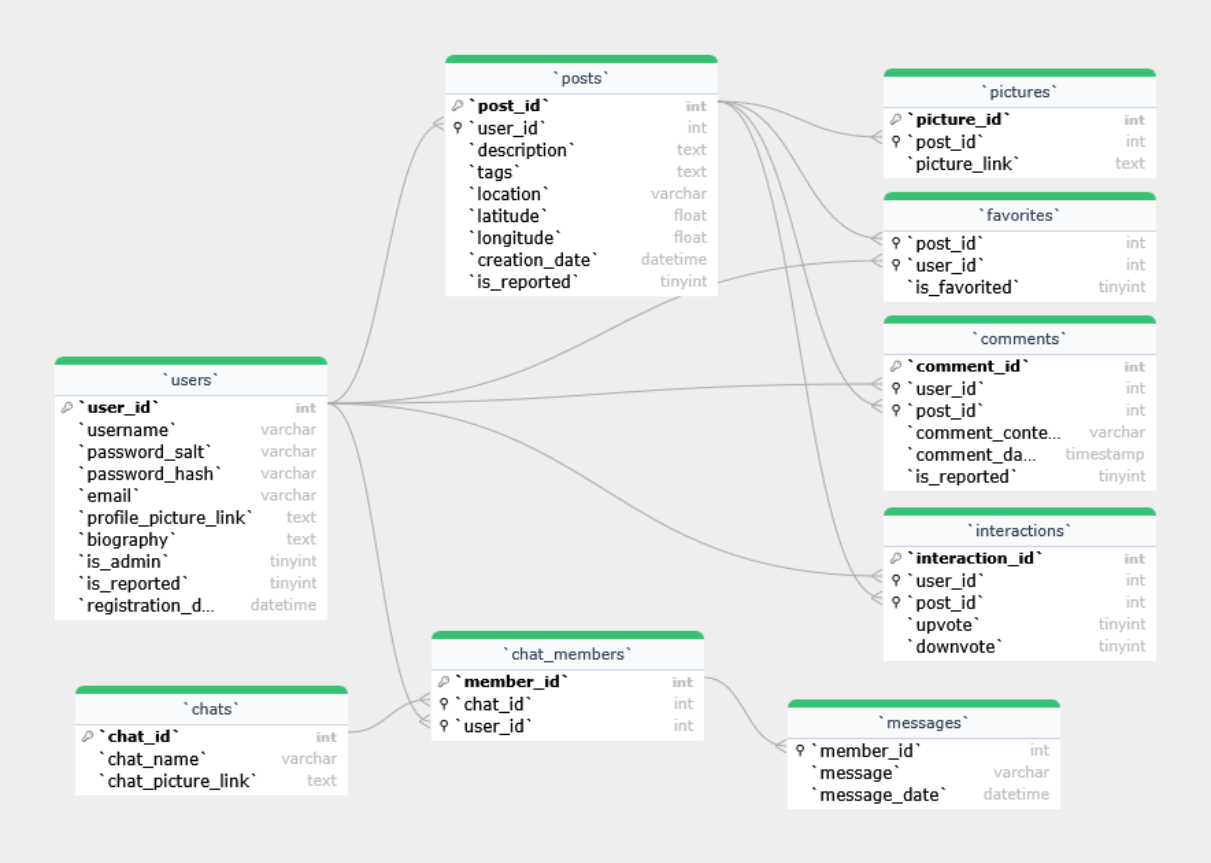
Ez a dokumentáció a „TELEPHOTO” nevű közösségi oldalról szól. Az alkalmazás elérhető webes felületen, mind asztali, mind hordozható számítógépről, illetve mobileszközökről is. A weboldalon egy kellemes közösségi élményt kínálunk kép- és videómegosztással, kedvencek kiválasztásával, új alkotások felfedezésével és felhasználók közötti beszélgetésekkel. Az oldal egyedisége, hogy a GPS-koordinátákkal ellátott képeket elhelyezzük egy térképen, így a felhasználók láthatják, hogy honnan származnak a képek. Emellett a felhasználók szűrhetik a térképen megjelenő posztokat, hogy csak azokat lássák, amelyekre kíváncsiak. A főoldalon a felhasználók a posztokat népszerűség, véletlenszerű helyszínek és keresett kulcsszavak alapján böngészhetik. Mindenkinek van egy profilja, ahol személyre tudjuk szabni a profilképünket, felhasználónevünket (amennyiben az elérhető) és szabadon írhatunk bemutatkozást.

Azért hoztuk létre ezt a projektet, mert kihívást akartunk állítani magunk elé, és egy hozzánk közel álló, érdekes témának találtuk egy közösségi média oldal létrehozását. Arra gondoltunk, hogy jó lehet saját preferenciánk és ötleteink alapján felépíteni egy ilyen oldalt. Sok ötletünk volt, és ez a projekt megengedte, hogy egy igazán egyedi terméket hozzunk létre. Úgy éreztük, egy ilyen projekttel sok új ismeretre tehetünk szert, illetve az iskolában tanult tudásunk minél nagyobb részét kamatoztatni tudjuk.

2 Fejlesztői dokumentáció

2.1 Adatbázis

A „Telephoto” adatbázisának feladata a posztok, posztokkal való interakciók, posztokhoz tartozó fájl referenciák, posztokhoz tartozó kommentek, beszélgetési információk, beszélgetésekhez tartozó tagok, beszélgetés üzenetei, és a felhasználók tárolása.



1. kép - Az adatbázis modellje

2.1.1 Technikai környezet

- Karakterkészlet: UTF-8
- Illesztési szabály: utf8_hungarian_ci
- Tárolt adatok: felhasználók, posztok, kommentek, interakciók, beszélgetések, fájl referenciák.

2.1.2 Kapcsolatok

1. Táblázat	2. Táblázat	Kapcsolat típusa	Leírás
users	posts	1:N	Egy felhasználónak több posztja is lehet
users	interactions	1:N	Egy felhasználó több poszttal is interaktálhat
users	comments	1:N	Egy felhasználó több poszthoz is kommentelhet
users	chat_members	1:N	Egy felhasználó több beszélgetéshez is tartozhat
posts	pictures	1:N	Egy poszthoz több kép is tartozhat
posts	comments	1:N	Egy poszthoz több komment is tartozhat
posts	interactions	1:N	Egy poszthoz több interakció is tartozhat
chats	chat_members	N:M	Egy beszélgetéshez több felhasználó tartozhat, és egy felhasználó több beszélgetéshez tartozhat
chat_members	messages	1:N	Egy felhasználóhoz több üzenet tartozhat

1. táblázat - Az adatbázis kapcsolatainak táblázata

2.1.3 Táblastruktúrák

2.1.3.1 chats

Mezőnév	Adattípus	Lehet NULL	Kulcs	Egyéb
chat_id	INT(11)	NEM	PRIMARY KEY	AUTO_INCREMENT
chat_name	VARCHAR(50)	NEM	-	-
chat_picture_link	TEXT	NEM	-	-

2. táblázat - A chats táblának a mezői

2.1.3.2 users

Mezőnév	Adattípus	Lehet NULL	Kulcs	Egyéb
user_id	INT(11)	NEM	PRIMARY KEY	AUTO_INCREMENT
username	VARCHAR(50)	NEM	-	-
password_salt	VARCHAR(32)	NEM	-	-
password_hash	VARCHAR(128)	NEM	-	-
email	VARCHAR(100)	NEM	-	-
profile_picture_link	TEXT	IGEN	-	-
biography	TEXT	IGEN	-	-
is_admin	TINYINT(1)	IGEN	-	DEFAULT 0
is_reported	TINYINT(1)	IGEN	-	DEFAULT 0
registration_date	DATETIME	IGEN	-	-

3. táblázat - A users táblának a mezői

2.1.3.3 posts

Mezőnév	Adattípus	Lehet NULL	Kulcs	Egyéb
post_id	INT(11)	NEM	PRIMARY KEY	AUTO_INCREMENT
user_id	INT(11)	NEM	FOREIGN KEY Kapcsolat: users(user_id)	-
description	TEXT	IGEN	-	-
tags	TEXT	IGEN	-	-
location	VARCHAR(176)	IGEN	-	-
latitude	FLOAT	IGEN	-	-
longitude	FLOAT	IGEN	-	-
creation_date	DATETIME	IGEN	-	-
is_reported	TINYINT(1)	IGEN	-	DEFAULT 0

4. táblázat - A posts táblának a mezői

2.1.3.4 pictures

Mezőnév	Adattípus	Lehet NULL	Kulcs	Egyéb
picture_id	INT(11)	NEM	PRIMARY KEY	AUTO_INCREMENT
post_id	INT(11)	NEM	FOREIGN KEY Kapcsolat: posts(post_id)	-
picture_link	TEXT	NEM	-	-

5. táblázat - A pictures táblának a mezői

2.1.3.5 interactions

Mezőnév	Adattípus	Lehet NULL	Kulcs	Egyéb
interaction_id	INT(11)	NEM	PRIMARY KEY	AUTO_INCREMENT
user_id	INT(11)	NEM	FOREIGN KEY Kapcsolat: users(user_id)	-
post_id	INT(11)	NEM	FOREIGN KEY Kapcsolat: posts(post_id)	-
upvote	TINYINT(1)	IGEN	-	-
downvote	TINYINT(1)	IGEN	-	-

6. táblázat - A interactions táblának a mezői

2.1.3.6 favorites

Mezőnév	Adattípus	Lehet NULL	Kulcs	Egyéb
user_id	INT(11)	NEM	FOREIGN KEY Kapcsolat: users(user_id)	-
post_id	INT(11)	NEM	FOREIGN KEY Kapcsolat: posts(post_id)	-
is_favorited	TINYINT(1)	IGEN	-	-

7. táblázat - A favorites táblának a mezői

2.1.3.7 comments

Mezőnév	Adattípus	Lehet NULL	Kulcs	Egyéb
comment_id	INT(11)	NEM	PRIMARY KEY	AUTO_INCREMENT
user_id	INT(11)	NEM	FOREIGN KEY Kapcsolat: users(user_id)	-
post_id	INT(11)	NEM	FOREIGN KEY Kapcsolat: posts(post_id)	-
comment_content	VARCHAR(150)	NEM	-	-
comment_date	TIMESTAMP	NEM	-	DEFAULT current_timestamp()
is_reported	TINYINT(1)	IGEN	-	-

8. táblázat - A comments táblának a mezői

2.1.3.8 chat_members

Mezőnév	Adattípus	Lehet NULL	Kulcs	Egyéb
member_id	INT(11)	NEM	PRIMARY KEY	AUTO_INCREMENT
chat_id	INT(11)	NEM	FOREIGN KEY Kapcsolat: chats(chat_id)	-
user_id	INT(11)	NEM	FOREIGN KEY Kapcsolat: users(user_id)	-

9. táblázat - A chat_members táblának a mezői

2.1.3.9 messages

Mezőnév	Adattípus	Lehet NULL	Kulcs	Egyéb
member_id	INT(11)	IGEN	FOREIGN KEY Kapcsolat: chat_members(member_id)	AUTO_INCREMENT
message	VARCHAR(200)	NEM	-	-
message_date	DATETIME	IGEN	-	-

10. táblázat - A messages táblának a mezői

2.1.4 Eseményindítók

Az adatbázis tartalmaz két darab eseményindítót. A „check_chat_for_delete” ami azt vizsgálja, hogy egy chathez tartozik-e legalább két darab chat_member, amennyiben nem, úgy azt a chatet törli, ez chat_members-en való törlés után fut le. A másik a „delete_user_member”, ez a users táblában való törlés előtt fut le, ami azért felel, hogy a törlendő felhasználó user_id-ja alapján törli a chat_members táblából a megfelelő sorokat.

2.2 Használt technológiák

Frontend:

- HTML
- CSS
- JavaScript
- Leaflet.js
- Leaflet.markercluster
- Socket.io-client
- ExifReader

Backend:

- Node.js + crypto modul + fs
- Express
- Socket.io
- Multer
- MySQL2

A Leaflet.js könyvtárt használjuk térképek és az azon található jelzők generálásához, a Leaflet.markercluster-t pedig a jelzők csoportosítására. Az ExifReader nyeri ki a fájlokból a

GPS adatokat. A Socket.io-t 3 fő működéshez használjuk, a beszélgetéseknél élőben frissüljenek az üzenetek, az admin oldalon az aktivitási grafikonhoz és amennyiben egy admin töröl egy felhasználót úgy azt a felhasználót jelentkeztesse ki a rendszer. A crypto modul segítségével titkosítjuk a jelszavakat. Az fs felel a felesleges fájlok törléséért. A MySQL2 segítségével érjük el az adatbázist.

2.3 Kiemelendő Algoritmusok

2.3.1 Backend

2.3.1.1 api.js

2.3.1.1.1 HashString függvény

A függvény paraméternek egy string-et vár. Először létrehozunk egy „salt” konstanst amiben a „cypto” node.js module egyik beépített függvényével (crypto.randomBytes) egy 32 karakter hosszú random string-et mentünk el hexadecimális számsorként.

Ezután egy „hash” konstanst hozunk létre, ami szintén beépített függvény (crypto.scryptSync) segítségével felhasználja a függvénybe paraméterként érkező string-et és a generált „salt”-ot, hogy egy 128 karakter hosszú stringet alkosson, amit hexadecimális számsorként adunk vissza a függvény végén a „salt”-al együtt. Ez biztosítja, hogy ha két felhasználónak ugyanaz lenne a jelszava, az adatbázisban ne ugyanúgy szerepeljenek.

```
//? Hash-eljük a megadott stringet, és visszaadunk egy salt, és egy hash változót.
function HashString(string) {
    //? salt: egy 32 karakteres random string amit átkonvertálunl hex-é
    const salt = crypto.randomBytes(16).toString('hex');
    console.log(salt);
    //? hash: a string, és salt alapján generált 128 karakteres string amit átkonvertálunk hex-é
    const hash = crypto.scryptSync(string, salt, 64).toString('hex');
    console.log(hash);
    return { salt, hash };
}
```

2. kép - HashString függvény kódrészlete

Ezt a függvényt a jelszavak titkosításához használjuk regisztrációkor a „/registration” api végpontban.

2.3.1.1.2 VerifyHashedString függvény

A HashString párja, paramáterként az ellenőrzendő stringet, és az eltárolt „salt”-ot és „hash”-t várja. A „/login” végpontban használjuk fel az eltárolt jelszóval való összevetéshez. Amennyiben nincs üres paraméter, azt vizsgálja, hogy ugyanazon „salt”

használatával a beírt string eredménye megegyezik-e a „hash”-el. Visszatérési értéknek igazat, vagy hamis-t ad.

```
function VerifyHashedString(string, salt, hash) {
    if (string !== null && salt !== null) {
        const hashedString = crypto.scryptSync(string, salt, 64).toString('hex');
        return hashedString === hash;
    } else {
        return null;
    }
}
```

3. kép - VerifyHashedString függvény kódrészlete

2.3.1.1.3 postTypeOffset objektum és /setOffset végpont

Egy megosztott szerveroldali objektum { offset: number, type: string }, amely nyomon követi a lapozás állapotát. A setOffset végpont vagy nullára állítja vissza (új keresés/szűrő esetén), vagy a köteg méretével (50) növeli az eltolást. Ez egy állapotalapú trükk, amely egyszerűvé teszi a klienst – nem kell nyomon követnie, hogy hány bejegyzés lett betöltve.

```
const postTypeOffset = {
    offset: 0,
    type: ''
};

router.post('/setOffset', async (request, response) => {
    const { type, offset } = request.body;
    if (type == 'reset') {
        postTypeOffset.offset = 0;
        postTypeOffset.type = 'reset';
    } else {
        postTypeOffset.type = type;
        postTypeOffset.offset += offset;
    }

    response.status(200).json({ status: 'success', result: postTypeOffset });
});
```

4. kép - A postTypeOffset objektum felépítése, és a /setOffset végpont kódrészlete

2.3.1.2 database.js

2.3.1.2.1 topPosts függvény

Az elsődleges lekérdezés, ami feltölti a főoldalt posztokkal, pontszámuk szerint csökkenő sorrendben. Összekapcsolja a bejegyzéseket, felhasználókat és interakciókat, időablak szerint szűr (NOW() - INTERVAL ? DAY), csoportosítja a bejegyzéseket, nettó pontszám szerint rendezi őket (SUM(upvote) - SUM(downvote) DESC), és LIMIT 50 OFFSET ? értékkel tölti fel az oldalt megfelelő egységenként. A lekérés után az eredményeket a getAllInteractions és a postPictures metódusokon keresztül továbbítja, hogy egyetlen hívásban teljesen dúsított bejegyzésobjektumokat hozzon létre.

```
async function topPosts(userId, timeFrame, offset) {
    try {
        let result;

        const topPostsSql = `
        SELECT posts.post_id, posts.description, posts.tags, posts.location, posts.latitude, posts.longitude, posts.creation_date, users.username, users.profile_picture_link, SUM(interactions.
        upvote) as upvote, SUM(interactions.downvote) as downvote
        FROM posts
        LEFT JOIN users ON users.user_id = posts.user_id
        LEFT JOIN interactions ON interactions.post_id = posts.post_id
        WHERE posts.creation_date > NOW() - INTERVAL ? DAY
        GROUP BY posts.post_id
        ORDER BY SUM(interactions.upvote) - SUM(interactions.downvote) DESC
        LIMIT 50 OFFSET ?;
        `;
        const [rows] = await pool.execute(topPostsSql, [timeFrame, offset]);
        result = await getAllInteractions(userId, rows);
        return await postPictures(result);
    } catch (error) {
        console.error(error);
    }
}
```

5. kép - topPosts függvény kódrészlete

2.3.2 Frontend

2.3.2.1 admin.js

2.3.2.1.1 generateBarChart

Ez a függvény felel az aktivitási grafikon generálásáért. Paraméterként kap két tömböt, az egyik a felhasználók - adott időpontokkor - mért számát tartalmazza, míg a másik az időpontokat hordozza magában. Egy ciklussal negyedekre osztjuk a közelmúlt csúcsforgalmának számát, az oszlopdiagram y-tengelyének mérőszámaihoz. Létrehozzuk a DOM elemeket, végig iteráljuk a felhasználók tömbjét, és beállítjuk az elemek magasságát a tömb számai alapján, ahol a legmagasabb oszlop a csúcsforgalom időpontjánál szerepel, a többi ehhez viszonyul. Végül megjelenítjük őket az oldalon.

```
export function generateBarChart(userTrackerArray, userTrackerTimeArray) {
    const yAxis = document.getElementById('yAxis');
    yAxis.replaceChildren();
    const nullPoint = document.createElement('span');
    nullPoint.innerText = 0;
    yAxis.appendChild(nullPoint);

    let maxValue = Math.max(...userTrackerArray);

    for (let i = 1; i < 5; i++) {
        if ((maxValue * (i / 4)) % 1 == 0) {
        }
        const span = document.createElement('span');
        span.innerText = maxValue * (i / 4);
        yAxis.appendChild(span);
    }

    let ratio = 100 / maxValue;
    let columnDiv = document.getElementById('userTrackerColumns');
    columnDiv.replaceChildren();
    let timeDiv = document.getElementById('userTrackerTimes');
    timeDiv.replaceChildren();
    for (let i = 0; i < userTrackerArray.length; i++) {
        let column = document.createElement('div');
        column.classList.add('userTrackerColumn');
        column.style.height = ratio * userTrackerArray[i] + '%';

        let tooltip = document.createElement('span');
        tooltip.classList.add('userTrackerTooltip');
        tooltip.innerText = userTrackerArray[i];
        if (ratio * userTrackerArray[i] < 40) {
            tooltip.style.paddingTop = '5%';
            tooltip.style.fontSize = '0.6vw';
        } else {
            tooltip.style.fontSize = '0.85vw';
        }

        column.appendChild(tooltip);
        columnDiv.appendChild(column);

        let span = document.createElement('span');
        span.innerText = userTrackerTimeArray[i];
        timeDiv.appendChild(span);
    }
}
```

6. kép - A generateBarChart kódrészlete

2.3.2.2 registration.js

2.3.2.2.1 registration

A registration.js-ben található registration függvényünk új felhasználói fiók létrehozását kezeli. Több ellenőrzést is elvégez mielőtt végrehajtja a kérést. Hogy sikeresen megtörténjen a regisztráció, megnézzük van-e beírt felhasználónév, megfelelő-e annak a hossza és az foglalt-e. Ezután regex segítségével a jelszókövetelmények teljesítését vizsgáljuk. Ez legalább 1 különleges karaktert, számot, illetve 8 darab karaktert jelent. Végül az ismételt jelszó sikeres összehasonlítása esetén regisztrál egy új fiókot. Bármelyik ellenőrzés hibája esetén ezt jelzi a felhasználónak vizuálisan.

```javascript
export async function registration() {
    try {
        let validInputs = 0;
        let errorMessage = '';
        let usernameInput = document.getElementById('registrationUsername');
        let username = usernameInput.value;
        if (username != '' && username.replace(/\s/g, '').length >= 3) {
            const isUsernameAvailable = await GetMethodFetch(
                '/api/isUsernameAvailable/' + username
            );
            if (isUsernameAvailable.available) {
                usernameInput.classList.remove('invalid');
                validInputs++;
            } else {
                usernameInput.classList.add('invalid');
                errorMessage += 'Ez a felhasználónév már foglalt!\n';
                validInputs--;
            }
        } else {
            errorMessage += 'Nem elég hosszú felhasználónév!\n';
            usernameInput.classList.add('invalid');
            validInputs--;
        }

        //? Email formátum
        //? [A-Za-z0-9]: minden nagybetű a-z-ig, minden kisbetű a-z-ig és minden szám
        //? \char: tartalmaznia kell az adott karaktert
        const emailRegExp = /^[A-Za-z0-9]+@[A-Za-z0-9]+\.[A-Za-z0-9]/;
        let emailInput = document.getElementById('registrationEmail');
        let email = emailInput.value;
        if (emailRegExp.test(email)) {
            emailInput.classList.remove('invalid');
            validInputs++;
        } else {
            errorMessage += 'Nem megfelelő e-mail formátum!\n';

            emailInput.classList.add('invalid');
            validInputs--;
        }
```

7. kép - A registration kódrészlete 1/2

```javascript
        let passwordInput = document.getElementById('registrationPassword');
        let password = passwordInput.value;
        let notSpecialCharacters = /[^a-zA-Z0-9áéíóúöüÁÉÍÓÚŐŰ]/;
        let numbers = /[0-9]/;
        if (
            password &&
            notSpecialCharacters.test(password) &&
            numbers.test(password) &&
            password.length >= 8
        ) {
            passwordInput.parentNode.classList.remove('invalid');
            validInputs++;
        } else {
            errorMessage += 'A jelszó nem elég erős!\n';
            passwordInput.parentNode.classList.add('invalid');
            validInputs--;
        }

        let passwordConfirmInput = document.getElementById('registrationPasswordConfirm');
        let passwordConfirm = passwordConfirmInput.value;
        if (passwordConfirm && passwordConfirm === password) {
            passwordConfirmInput.parentNode.classList.remove('invalid');
            validInputs++;
        } else {
            errorMessage += 'A két jelszó nem egyezik meg!\n';

            passwordConfirmInput.parentNode.classList.add('invalid');
            validInputs--;
        }

        if (validInputs === 4) {
            const response = await PostMethodFetch('/api/registration', {
                username: username,
                email: email,
                password: password
            });
            emailInput.value = '';
            usernameInput.value = '';
            passwordConfirmInput.value = '';
            passwordInput.value = '';
            if (response.status == 'Successful registration') {
                return { success: true };
            } else {
                return { success: false, errorMessage: errorMessage };
            }
        } else {
            return { success: false, errorMessage: errorMessage };
        }
    } catch (error) {
        console.error(`REGISZTRÁCIÓS hiba: ${error.message}`);
    }
}
```

8. kép - A registration kódrészlete 2/2

2.3.2.3 uploadPost.js

2.3.2.3.1 uploadPost

Ez a függvény felelős a posztok feltöltéséért. A függvény elsőkörben megszerzi a kiválasztott fájlokat, amennyiben a fájlok száma nem 0 és a fájl formátumot ellenőrző „rightFileFormats” függvény is igaz értéket ad vissza úgy folytatódik a függvény. Ezután megvizsgáljuk az ExifReader segítségével, hogy a legelső fájlnak vannak-e GPS koordinátái, majd végig iteráljuk a fájlokat és azok alapján létrehozunk újakat, átnevezve, erre azért van szükség, hogy kiszűrjünk néhány speciális karaktert például a ’.’-t. Megszerezzük a kitöltött mezők értékeit, illetve az „addedTag” osztállyal ellátott elemeket. Ezeknek az elemeknek a tartalmából készítünk egy string-et, hogy eltudjuk tárolni az adatbázisban. Végül meghívjuk az „/uploadPost” és a „/createPost” végpontokat és bezárjuk a felugró ablakot.

```
export async function uploadPost() {
    try {
        let files = document.getElementById('uploadFile').files;
        if (files.length > 0 && rightFileFormats(files)) {
            let gps;
            (await getGPS(files[0])) == null
                ? (gps = { Latitude: null, Longitude: null })
                : (gps = await getGPS(files[0]));

            let renamedFiles = [];
            for (let file of files) {
                let renamedFile = new File(
                    [file],
                    file.name
                        .normalize('NFD')
                        .replace(/[\u0300-\u036f]/g, '')
                        .replace(/\.(?=.*\.)/g, '-')
                        .replace(/\(/g, '')
                        .replace(/\)/g, '')
                        .replace(' ', ''),
                    {
                        type: file.type
                    }
                );
                renamedFiles.push(renamedFile);
            }

            let description = document.getElementById('uploadDescription').value;
            let location = document.getElementById('uploadLocation').value;
            let tags = document.querySelectorAll('.addedTag');
            let uploadTags = '';
            for (let tag of tags) {
                uploadTags += tag.innerText + ' ';
            }
            const usernameResponse = await GetMethodFetch('/api/sendUsername');
            const uploadResponse = await uploadFiles('/api/uploadPost', renamedFiles);
            let uploadedFiles = [];
            for (const object of uploadResponse.filenames) {
                uploadedFiles.push(object.filename);
            }

            const createPostResponse = await PostMethodFetch('/api/createPost', {
                username: usernameResponse.Result,
                fileNames: uploadedFiles,
                description: description,
                tags: uploadTags,
                location: location,
                latitude: gps.Latitude,
                longitude: gps.Longitude
            });

            if (createPostResponse.Success) {
                uploadClose();
            }
        } else {
            document.getElementById('uploadFileLabelText').classList.add('invalid');
            document.getElementById('tempSlideshowWrapper').classList.add('invalid');
        }
    } catch (error) {
        console.error(error);
    }
}
```

9. kép - Az uploadPost kódrészlete

2.3.2.4 post.js

2.3.2.4.1 hangPictures

Ez a függvény felelős a posztok generálásáért. Paraméterként kap egy objektumot, amit korábban már az adatbázisból lekértünk. Ezután létrehozunk változókat az oldalon megjelenő vizuális elemeknek. A segédfüggvények megkapják az objektum értékeit, amik így legenerálják a DOM elemeket, majd ezután az „appendChild" paranccsal felépítjük a posztot. A függvény végén hozzárendelünk a poszthoz egy „IntersectionObserver" -t, ami a poszt képernyőre kerülése esetén hozzárendel egy animációt.

```
const hangPictures = async (test) => {
    let posts = document.getElementById('posts-container');
    let post = document.createElement('div');
    post.classList.add('post');

    let ropediv = generateRope();
    let clip = generateClip();
    let slideshow = utilFunctions.generateSlideshow('/uploads/', test.links);

    let timestamp = utilFunctions.generateTimestamp(test.creation_date);
    slideshow.appendChild(timestamp);

    let tags = utilFunctions.generateTags(test.tags, test.location);

    let description = utilFunctions.generateDescription(test.description);

    let interactionRow = await utilFunctions.generateInteractions(
        test.interactions[0].like,
        test.interactions[0].dislike,
        test.interactions[0].favorite,
        test.post_id,
        test.upvote,
        test.downvote
    );

    let userRow = utilFunctions.generateUserRow(test.username, test.profile_picture_link);

    let postcontent = document.createElement('div');
    postcontent.classList.add('postContent');

    postcontent.appendChild(slideshow);

    postcontent.appendChild(interactionRow);
    postcontent.appendChild(userRow);
    postcontent.appendChild(tags);
    postcontent.appendChild(description);

    post.appendChild(ropediv);
    post.appendChild(clip);
    post.appendChild(postcontent);
    post.dataset.postId = test.post_id;
```

10. kép - A hangPictures kódrészlete 1/2

```
    const options = {
        root: posts,
        threshold: 0
    };

    const callback = (entries) => {
        let matchMedia = window.matchMedia('(width > 1000px)');

        entries.forEach((element) => {
            if (element.isIntersecting) {
                let postContent = element.target.children[2];
                if (matchMedia.matches) {
                    postContent.style.animation = 'fadeInUp 0.5s forwards';
                } else {
                    postContent.style.animation = 'fadeInLeft 0.5s forwards';
                }
            }
        });
    };
    const observer = new IntersectionObserver(callback, options);

    observer.observe(post);

    posts.appendChild(post);
};
```

11. kép - A hangPictures kódrészlete 2/2

2.3.2.5 chat.js

2.3.2.5.1 getChatData

A főoldalon látható chatek listájához szükséges listát hozza létre. A „/sendUsername” meghívásával lekérdezi az oldalt használó user nevét, majd ezt felhasználva kérést küld a „/chatsOfUser/:username”-nek. A lekérdezés eredményein egy for of ciklust indítunk el, amely öszeállítja a „ChatData” példányokat. Ebbe belekerül: obj.chat_id, obj.chat_name, obj.chat_picture_link, és a „/lastMessageOfChat/:chatId” eredménye, azaz az utolsó üzenet a chatben. A példányok végül mind belekerülnek a függvény elején létrehozott „chatArray”-be, ami végül a visszatérési érték is.

```
export async function getChatData() {
    try {
        let chatArray = [];
        let username = await GetMethodFetch('/api/sendUsername');
        username = username.Result;
        const chatsResponse = await GetMethodFetch('/api/chatsOfUser/' + username);
        const result = chatsResponse.Result;
        for (const obj of result) {
            const lastMessageResponse = await GetMethodFetch(
                '/api/lastMessageOfChat/' + obj.chat_id
            );
            if (lastMessageResponse.Status == 'Success') {
                const chat = new ChatData(
                    obj.chat_id,
                    obj.chat_name,
                    obj.chat_picture_link,
                    lastMessageResponse.Result == undefined
                        ? ''
                        : lastMessageResponse.Result.message,
                    lastMessageResponse.Result == undefined
                        ? ''
                        : lastMessageResponse.Result.username
                );
                chatArray.push(chat);
            }
        }

        return chatArray;
    } catch (error) {
        throw new Error(error);
    }
}
```

12. kép - A getChatData kódrészlete

2.3.2.6 map.js

2.3.2.6.1 filterPins

Ennek a függvénynek a segítségével tudjuk szűrni a posztokat hosszúság és szélesség szerint. A függvény létrehoz egy „val” nevű változót, ami végigmegy az adott inputnak az értékén. A „val” változó csak akkor kapja meg a beírt értéket, ha az megfelel a követelményeknek (nem betű, nem lehet több negatív jel, nem lehet több tizedespont, maximum értéke 90 vagy 180 kontextus függvényében, a minimum értéke -90 vagy -180 a kontextus függvényében). Miután összeállt a „val” értéke megvizsgálja, hogy a minimum és maximum érték érvényes-e és amennyiben igen, újra generálja a „marker”-eket a térképen.

```
export function filterPins() {
    let val = '';
    let dot = false;
    for (let i = 0; i < this.value.length; i++) {
        if (i == 0 && this.value[i] == '-') {
            val += this.value[i];
        }
        if (/[0-9]/.test(this.value[i])) {
            val += this.value[i];
        }
        if (!dot && this.value[i] == '.') {
            dot = true;
            if (i == 0 || (i == 1 && this.value[0] == '-')) {
                val += '0' + this.value[i];
            } else {
                val += this.value[i];
            }
        }
    }

    if (this.classList.contains('filterLat')) {
        if (val > 90) val = 90;

        if (val < -90) val = -90;
    }

    if (this.classList.contains('filterLon')) {
        if (val > 180) val = 180;

        if (val < -180) val = -180;
    }
    this.value = val;

    let min = this.parentNode.children[0];
    let max = this.parentNode.children[1];
    if (val != '-' && val != '.') {
        if (parseFloat(max.value) < parseFloat(min.value) && this == max) {
            max.classList.add('invalidFilter');
            min.classList.remove('invalidFilter');
        } else if (parseFloat(max.value) < parseFloat(min.value) && this == min) {
            max.classList.remove('invalidFilter');
            min.classList.add('invalidFilter');
        } else {
            max.classList.remove('invalidFilter');
            min.classList.remove('invalidFilter');
            markerCluster.clearLayers();
            map.removeLayer(markerCluster);

            generateFilterLines(this, max);
            generateFilteredMarkers();
        }
    }
}
```

13. kép - A filterPins kódrészlete

2.4 Tesztelés

2.4.1 Bevezetés

Ez a fejezet a „Telephoto” közösségi oldal tesztelési folyamatát, környezetét és eredményeit tartalmazza. A tesztelés, egységtesztek formájában valósult meg. A célja a különféle függvények eredményeinek ellenőrzése. A tesztelés a „Jest” keretrendszerével készült, ez lehetővé teszi az API-ok és socket kapcsolatok mockolását.

2.4.2 Technikai környezet

- Jest (^30.3.0): Maga a keretrendszer, ennek segítségével fut a tesztelést.
- babel-jest (^30.3.0): Ez alakítja át az ES modulokat CommonJS modulokká.
- jest-environment-jsdom (^30.3.0): Ennek segítségével szimuláljuk a DOM elemeket.
- Node.js (v10.9.2)

2.4.3 Tesztesetek

2.4.3.1 util.js

Teszteljük az általános használt függvényeket. Tartalmaz dátum formázó, bejelentkezés ellenőrző, admin jogosultság ellenőrző és DOM elem generáló függvényeket.

Függvény	Eset	Bemenet	Elvárt kimenet	Státusz
date_yyyy_MM_dd	Általános érték	Mar 25 2015	2015-03-25	Átment
	Határ érték	Dec 31 2015	2015-12-31	Átment
	Üres érték	(üres string)	(üres string)	Átment
date_yyyy_MM_dd_hh_mm	Általános érték	2026-04-17 12:26:57	2026-04-17 12:26	Átment
	Határ érték	Dec 31 2020 23:59	2020-12-31 23:59	Átment
	Üres érték	(üres string)	(üres string)	Átment
isLoggedIn	A felhasználó be van jelentkezve	-	true	Átment
	A felhasználó nincs bejelentkezve	-	false	Átment
	API hívás hiba	-	Error(„Hiba az "/sendUsername" végpontban”)	Átment
isAdmin	A felhasználó admin	-	true	Átment
	A felhasználó nem admin	-	false	Átment
	API hívás hiba	-	Error(„Hiba az "/isAdmin" végpontban”)	Átment

generateSlideshow	Általános értékekkel	„link1.png”, „link2.mp4”	Legenerálja a tartalmat és a léptetőket. A tartalmat tartalmazó szülő elemnek a gyermekeinek száma megegyezik bemeneti tömb hosszával	Átment
	Üres értékekkel	(üres tömb)	A szülő elemnek nincsen gyermeke	Átment
	Üres értékekkel	(undefined)	A szülő elemnek nincsen gyermeke	Átment
generateTimestamp	Általános érték	2026-04-18T10:52:09.000Z	2026-04-18	Átment
	Üres érték	(üres string)	Az elem tartalma üres	Átment
	Üres érték	(undefined)	Az elem tartalma üres	Átment
generateTags	Általános értékek	„#tag1#tag2” , „location”	Legenerál minden elemet a megfelelő értékekkel	Átment
	Üres tag-ek, általános lokáció	(üres string), „location”	A lokáció elemét legenerálja, míg a tagek elemeit nem	Átment
	Általános tag-ek, üres lokáció	„#tag1#tag2”, (üres string)	A tag elemeket legenerálja, míg a lokációt nem	Átment

	Üres értékek	(üres string), (üres string)	Nem generál elemeket	Átment
generateDescription	Általános érték	„Template description”	A leírás tartalma a megadott érték legyen	Átment
	Üres érték	(üres string)	A leírás tartalma üres legyen	Átment
	Üres érték	(undefined)	A leírás tartalma üres legyen	Átment
generateInteractions	Általános értékek	1,0,1,1,100,10	Minden elem megkapta a megfelelő osztályokat	Átment
generateUserRow	Átlalános értékek	„felhasználónév”, „test.png”	Tartalmaz minden elemet, a kép hivatkozása tartalmazza a megadott értéket és a megadott érték megegyezik a generált névvel	Átment
	Általános felhasználónév, undefined profil kép	„felhasználónév”, (undefined)	Tartalmaz minden elemet, a kép hivatkozása tartalmazza az alapértelmezett értéket és a megadott érték megegyezik a generált névvel	Átment
	Általános felhasználóné	„felhasználónév”, (üres string)	Tartalmaz minden elemet, a kép	Átment

	v, (üres string) profil kép		hivatkozása tartalmazza az alapértelmezett értéket és a megadott érték megegyezik a generált névvel	
generateCommentProfilePicture	Általános értékek	„test.png”, „felhasználónév”	A kép hivatkozása tartalmazza a megadott értéket	Átment
	Üres profilkép	(üres string), „felhasználónév”	A kép hivatkozás tartalmazza az alapértelmezett kép hivatkozását	Átment
	Üres profilkép	(undefined), „felhasználónév”	A kép hivatkozás tartalmazza az alapértelmezett kép hivatkozását	Átment
generateCommentContent	Általános érték	„lorem ipsum”	Az elem tartalma a megadott érték legyen	Átment
	Üres érték	(üres string)	Az elem tartalma üres legyen	Átment
generateCommentUsername	Általános érték	„felhasználónév”	Az elem tartalma a megadott érték legyen	Átment

	Üres érték	(üres string)	Az elem tartalma üres legyen	Átment
generateCommentDate	Általános érték	2026-04-19 12:03:55	Az elem tartalma a megadott érték legyen az nap után sortöréssel	Átment
generateComment	Általános érték	{profile_picture_link: „test.png”, username: „felhasználónév”, comment_content: „lorem ipsum”, comment_date: „2026-04-19 12:03:55”}	A generált elemnek legyen két gyereke valamint a szöveges tartalmakat tartalmazó szülőnek legyen 3 gyereke	Átment

11. táblázat - A util.js tesztesetei

2.4.3.2 admin.js

Egy függvényt tesztelünk, amely unix timestampből óra perc stringet készít (ó:pp), és egy általános rendező függvényt.

Függvény	Eset	Bemenet	Elvárt kimenet	Státusz
convertToTime	Általános érték	1670414605000	13:03	Átment
	Határ érték	0	1:00	Átment
sortArray	Általános értékek	1,3,2,5,4	1,2,3,4,5	Átment
	Egy elemes tömb	1	1	Átment
	Általános értékek ismétléssel	2,1,1,3,2,5	1,1,2,2,3,5	Átment

12. táblázat - Az admin.js tesztesetei

2.4.3.3 registration.js

Egy regisztrációs függvényt tesztelünk, amely vizsgálja a bekért értékek helyességét és amennyiben megfelelnek elküldi azokat a „registration” végpontnak, majd a visszaérkező válasz függvényében visszaadunk egy objektumot ami tartalmaz egy „success” értéket, illetve ha valami probléma támad egy „errorMessage” értéket.

Függvény	Eset	Bemenet	Elvárt kimenet	Státusz
registration	Sikeres regisztráció	„felhasználónév”, „a@a.com”, „1234567.”, „1234567.”	{ success: true }	Átment
	Foglalt felhasználónév	„felhasználónév”, „a@a.com”, „1234567.”, „1234567.”	{ success: false, errorMessage: 'Ez a felhasználónév már foglalt!\n' }	Átment
	A két jelszó nem egyezik	„felhasználónév”, „a@a.com”, „12345678.”, „1234567.”	{ success: false, errorMessage: 'A két jelszó nem egyezik meg!\n}	Átment
	A jelszó nem elég erős	„felhasználónév”, „a@a.com”, „1”, „1”	{ success: false, errorMessage: ’A jelszó nem elég erős!\n ’ }	Átment
	Nem megfelelő az email formátum	„felhasználónév”, „aa.com”, „1234567.”, „1234567.”	{ success: false, errorMessage: Nem megfelelő e-mail formátum!\n ’ }	Átment
	Nem elég hosszú a felhasználónév	„a”, „a@a.com”, „1234567.”, „1234567.”	success: false, errorMessage: Nem elég hosszú felhasználónév!\n ’ }	Átment
	Foglalt felhasználónév, a két jelszó nem egyezik, a jelszó nem elég erős, nem megfelelő email formátum	„felhasználónév”, „aa.com”, „1”, „1234567.”	success: false, errorMessage: ’Ez a felhasználónév már foglalt!\nNem megfelelő e-mail formátum!\nA	Átment

			jelszó nem elég erős!\nA két jelszó nem egyezik meg!\n ' }	

13. táblázat - A registration.js tesztesetei

2.4.3.4 login.js

Egy bejelentkező függvényt tesztelünk, amely vizsgálja a bekért értékek helyességét és amennyiben megfelelnek elküldi azokat a „login" végpontnak, majd a visszaérkező válasz függvényében visszaadunk true vagy false értéket.

Függvény	Eset	Bemenet	Elvárt kimenet	Státusz
login	Sikeres bejelentkezés	„felhasználónév", „1234567."	true	Átment
	Sikertelen bejelentkezés	„felhasználónév", „1234567."	false	Átment
	Sikeretelen bejelentkezés, nem adott meg jelszót	„felhasználónév", (üres string)	false	Átment
	Sikeretelen bejelentkezés, nem adott meg felhasználónevet	(üres string), „1234567."	false	Átment

14. táblázat - A login.js tesztesetei

2.4.3.5 chat.js

Három függvényt tesztelünk, egy adatlekérőt ami visszaadja egy felhasználó beszélgetéseinek az adatait egy ChatData objektumban. Egy üzenet generálót, amely megkapja a felhasználó nevét, az üzenet küldő nevét, az üzenetet és az üzenetküldésének dátumát. Valamint egy üzenet beviteli mező generáló függvényt.

Függvény	Eset	Bemenet	Elvárt kimenet	Státusz
getChatData	Általános értékek (2db beszélgetés)	-	Egy tömb 2db ChatData objektum	Átment

	Határ érték (0db beszélgetés)	-	Egy üres tömb	Átment
	Hiba (nincs felhasználónév)	-	Error(„Hiba az "/sendUsername" végpontban”)	Átment
generateMessage	Sikeres generálás (a jelenlegi felhasználó küldte az üzenetet)	„felhasználónév”, „felhasználónév”, „üzenet”, „2026-04-18 12:33:07”	Az üzenet tartalmazza a megfelelő osztályokat	Átment
	Sikeres generálás (más felhasználó küldte az üzenetet)	„felhasználónév”, „másfelhasználónév”, „üzenet”, „2026-04-18 12:33:07”	Az üzenet tartalmazza a megfelelő osztályokat	Átment
generateMessageInput	Sikeres generálás	-	A generált DOM element gyermekeinek a száma 2	Átment

15. táblázat - A chat.js tesztesetei

2.4.3.6 uploadPost.js

Egy függvényt tesztelünk, amely megvizsgálja, hogy a bemenetben megadott fájloknak a formátuma helyes-e és visszaad egy igaz hamis értéket.

Függvény	Eset	Bemenet	Elvárt kimenet	Státusz
rightFileFormat	Minden fájl formátuma helyes	new File(['test'], 'test.png', { type: 'image/png'}), new File(['test'], 'test.mp4', { type: 'video/mp4'})	true	Átment
	1 fájlnak helytelen a formátuma	new File(['test'], 'test.txt, { type: text/txt'}), new File(['test'], 'test.mp4', { type: 'video/mp4'})	false	Átment
	Minden fájl formátuma hely	new File(['test'], 'test.txt, { type: text/txt'}), new File(['test'], 'test.mp4', { type: 'video/mp4'})	false	Átment

16. táblázat - Az uploadPost.js tesztesetei

2.4.3.7 *profile.js*

Két darab függvényt tesztelünk, az egyik fájl formátumot vizsgál majd visszaad egy igaz hamis értéket, a másik DOM generáló.

Függvény	Eset	Bemenet	Elvárt kimenet	Státusz
isImageFormat	Helyes formátum	File(['test'], 'test.png')	true	Átment
	Helytelen formátum	File(['test'], 'test.txt')	false	Átment
	Hiányzó fájl	(undefined)	false	Átment
generatePostDelete	Sikeres generálás	-	A visszaadott DOM elem innerHTML-e nem üres	Átment

17. táblázat - A profile.js tesztesetei

2.4.3.8 *map.js*

Egy lekérő függvényt tesztelünk, amely visszaadja azon posztokat, ahol a szélesség és hosszúsági fok nem NULL.

Függvény	Eset	Bemenet	Elvárt kimenet	Státusz
getMarkers	Sikeres lekérés	-	Egy tömb, amely az összes megfelelő objektumot tartalmazza	Átment
	Sikeres lekérés (0db)	-	Egy üres tömb	Átment

18. táblázat - A map.js tesztesetei

2.4.3.9 post.js

Két DOM generáló függvényt tesztelünk, az egyik egy „csipeszt”, a másik egy „kötelet” generál.

Függvény	Eset	Bemenet	Elvárt kimenet	Státusz
generateClip	Sikeres generálás	-	A visszaadott DOM elem tartalmaz két darab gyerek elemet	Átment
generateRope	Sikeres generálás	-	A visszaadott DOM elem tartalmaz két darab gyerek elemet	Átment

19. táblázat - A post.js tesztesetei

2.4.4 Teszt lefedettség

Fájl	Lefedettség
util.js	15 függvény 38 teszteset
admin.js	2 függvény 5 teszteset
registration.js	1 függvény 7 teszteset
login.js	1 függvény 4 teszteset
chat.js	3 függvény 5 teszteset
uploadPost.js	1 függvény 3 teszteset
profile.js	1 függvény 3 teszteset
map.js	1 függvény 2 teszteset
post.js	2 függvény 2 teszteset

20. táblázat - Teszt lefedettség

2.4.5 Teszt futtatása

2.4.5.1 Előfeltételek

- A csomagok telepítve vannak, amennyiben nincsenek ezt a „backend” mappából az „npm install” paranccsal tudjuk ezeket telepíteni, a konzol segítségével.
- babel.config.json

2.4.5.2 Futtatás

A tesztet a „backend” mappából futattjuk az „npm test” parancssal, konzol segítségével.

2.4.5.3 Eredmények értelmezése

A teszt lefuttatása után a konzolon láthatjuk az eredményt. A teszt esetek mellet szerepelni fog egy „pipa” vagy „x”. Amennyiben „pipa” szerepel a teszteset mellet úgy a teszteset sikeresen lefutott és a kimenet megegyezett az elvárt kimenettel. Viszont, ha „x” jelenik meg abban az esetben az adott teszteset sikertelen.

2.5 Továbbfejlesztési Lehetőségek

2.5.1 Élő Adás

Sok közösségi oldalon manapság már elérhető az a funkció, hogy élő adást indíthassunk. Ez egy érdekes extra lehetne a mi oldalunkon is. Valószínűleg egy gondolkodást serkentő projekt live streaming felületet lekodólni, azonban a mi időkeretünk, ezt már nem engedte meg, hogy az ehhez szükséges információkat megtanuljuk és meg is valósítsuk őket.

2.5.2 Videohívás

Egyre több olyan alkalmazásnak van videohívás funkciója, ahol a chat már meglévő eleme a programnak. Manapság a közösségi médiás felületek nagyrészén felhívhatjuk ismerőseinket, de banki vagy egyéb szolgáltatásoknál is van lehetőségünk video, vagy voicechat-en ügyeket intézni és problémákat jelenteni. Erre két megoldást is szoktak alkalmazni. Van ahol élő emberhez, van ahol AI ügyintézővel kapcsolnak minket össze. A mi oldalunkba is beleillenének a különböző hívási lehetőségek. Sajnos időnk már nem engedte, illetve még több utánajárást igényelt volna, de egy esetleges továbbfejlesztésnél jó ötlet lehetne implementálni hang és videokommunikációt felhasználók között, vagy akár adminokkal, segítő botokkal.

2.5.3 Eltűnő Posztok

A Meta platformjairól ismerősek lehetnek az ideiglenes, eltűnő posztok, másnéven story-k. Jól lehet őket használni kevésbé fontos, kevésbé átgondolt/szerkesztett dolgok megosztására vagy hírek megosztására. Esetlegesen a mi platformunkon is be lehetne mutatni egy frissítés keretében és a felhasználók például rövid helyzetjelentésekhez, egy nagyobb projektek előzeteséhez is használhatnák a funkciót. A mostani fejlesztési

időszak alatt mi nem éreztük úgy, hogy beleillene a mi stílusunkba, arculat tervünkbe, ahol nem akartunk mindenhová dopaminbombákat elhelyezni a felületünkre, ezzért döntöttünk az implementálása ellen.

2.5.4 Fotós Piactér

Egy egyedibb ötlet, amit több idő esetén implementálni lehetne a projektbe az a fotós piactér. Ennek az oldalnak több funkciója is lehetne. Az egyik, az egy szokásosabb használtpiac, ahol fotós kellékeket, eszközöket és kamerákat lehetne adni-venni. A másik része a marketplace oldalunknak a kép vásár lehetne. Itt Kereskedelmi használatra eladhatnák képeik használati jogát a felhasználók, vagy kereshetnének nekik szükséges képeket, akár egy stockfotó webáruházban.

3 Felhasználói dokumentáció

3.1 A program célja

A „TELEPHOTO” egy új közösségi oldal, mely elsősorban képek megosztására lett kifejlesztve. Átlátható és egyedi felületével igyekszik bevonni és megtartani az új felhasználók figyelmét. Az egyedi főoldal mellett térképes felületével is kíván eltérni vetélytársaitól.

3.2 A program funkciói

3.2.1 Regisztráció és Belépés

A weboldal felkeresésekor a belépési felület vár minket. Amennyiben még nincs fiókunk, ugyanitt regisztrálni is tudunk.

3.2.2 Főoldal

A főoldalon azonnal láthatóak a posztok, és az alattuk lévő kommentek is. Lehet a posztokat like-olni, kedvencek közé tenni, és új hozzászólást írni hozzájuk. A megjelenítésük sorrendjét lehet változtatni idő vagy népszerűség alapján is.

A főoldalon egyből elérhetőek a chatek is. Be lehet lépni más felhasználókkal indított, már élő beszélgetésekbe, és újakat is lehet indítani, egy-egy gombnyomással.

Más funkciókat a navbar használatával érhetünk el.

3.2.3 Posztolás

Szintén a főoldalon, a jobb alsó sarokban lévő kis ikonnal nyílik meg felugró ablakban a lehetőség, hogy új posztokat hozzunk létre. Itt feltölthetjük a kívánt képeket és csatolhatunk hozzá címet, leírást és helyszínt.

3.2.4 Térkép

A navbaron át elérhető a térkép. Itt a feltöltött posztok kitűzőcsoportokként jelennek meg a térképen, és nagyításkor egyre több jelenik meg. Itt rákattintva láthatjuk, hogy az adott helyen milyen kép készült.

3.2.5 Profil

A navbaron át érhető el a saját profil. Ezen az oldalon módosíthatók a személyes adatok, áttekinthetőek, és módosíthatóak a saját posztok. Kattintással a saját posztok helyén a like-olt, dislike-olt illetve elmentett kedvencek lesznek láthatóak.

3.2.6 Admin

Az platform adminjai számára elérhető az admin felület. Ez egy külön, átlagos felhasználók számára nem elérhető és nem látható oldal. Itt felügyelhetik az aktív felhasználók számát, és elbírálhatják a beérkezett jelentéseket is. Minden, a közösségi oldalon megjelenő tartalmat és profilt lehet moderálni, módosítani és törölni is.

3.3 Szükséges hardver és szoftvereszközök

3.3.1 Hardver

- Operációs rendszer: Windows 10 vagy újabb (32 és 64 bites)
- Processzor: Pentium 4 vagy újabb (SSE2 támogatással)
- RAM: 1 GB (32-bit) / 2 GB (64-bit)

3.3.2 Szoftver

- Modern webböngésző: Firefox, Google Chrome, Microsoft Edge

3.4 Telepítés és/vagy indítás részletei

- XAMPP elindítása
- MySql admin felület megnyitása és „database.sql” importálása
- VS Code-ban terminal indítása
- A „backend” mappában „npm install”, majd „npm run dev” parancsok beírása
- A weboldalt a http://127.0.0.1:3000/ linken keresztül tudjuk elérni

3.5 A program részletes bemutatója

3.5.1 Bejelentkezés és Regisztráció

Az oldal betöltésekor megjelenő felület, abban az esetben, ha nem vagyunk bejelentkezve. Ha már van fiókunk, akkor a helyes felhasználónév és jelszó kombinációjával be is léphetünk, és a főoldalra kerülünk.

Amennyiben még nincs fiókunk, rákattinthatunk a regisztráció gombra, ami felhozza az űrlapot a új felhasználó létrehozásához. Itt ki kell választanunk magunknak egy felhasználónevet, be kell írnunk egy érvényes e-mail címet, illetve egy jelszót amit meg is ismétlünk az elírás elkerülése végett. Az érvényes jelszónak tartalmaznia kell betűt, számot és speciális karaktert, valamint legalább 8 karakter hosszúnak kell lennie. Hiba esetén vizuálisan jelez az oldal, hogy elírtunk valamit.

14. kép - Bejelentkezési felület

15. kép - Regisztrációs felület

3.5.2 Navigációs sáv

Ezen keresztül tudunk lépkedni a főoldal, a térkép, a profil, és amennyiben van felhatalmazásunk, az admin felületre. Ez megtalálható a főoldalon, a térképen és a profilon is.

7. kép - Navigációs sáv

3.5.3 Főoldal

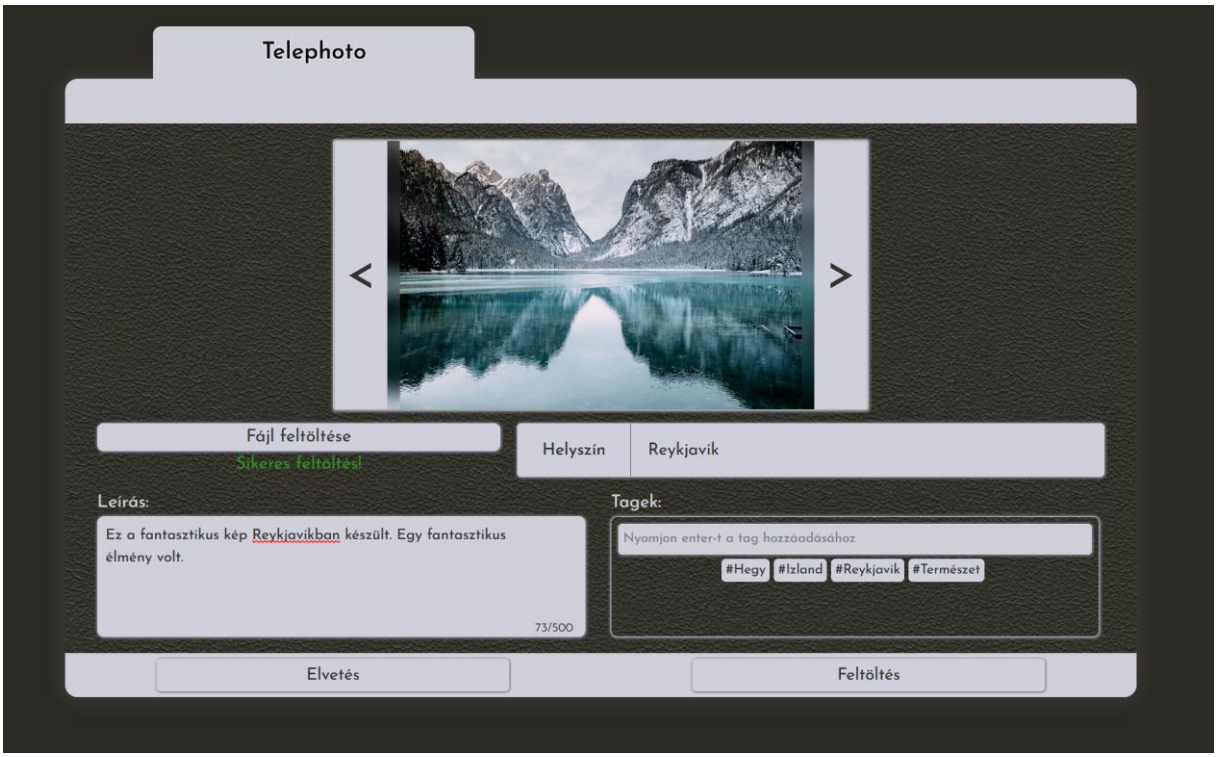
A főoldalt már megnyitáskor tartalommal tele látjuk. Középen egy vízszintesen görgethető felületen a posztokat láthatjuk. Alapértelmezetten minden idők legjobb posztjait like-ok szerint csökkenő sorrendben. Ezt a rendezést a bal oldali menüben állíthatjuk át a látható opciókkal, vagy akár szűkíthetjük a megjelenített posztok listáját a keresővel. A kereső alatt találhatóak a chat-jeink, illetve új beszélgetés indításának a lehetősége.

3.5.3.1 Posztolás

A posztolás gomb megnyomásával feljön az ablak új tartalom megosztásához. Itt fel tudunk tölteni képeket vagy videókat a saját eszközünkről a fájl feltöltése gombbal.

16. kép - Kitöltetlen feltöltési felület

A többi mezőben megadhatjuk a helyét, leírását, és címkéket is hozzárendelhetünk a poszthoz. Amennyiben az első kép tartalmaz GPS adatokat, úgy az oldal azokat is elmenti.



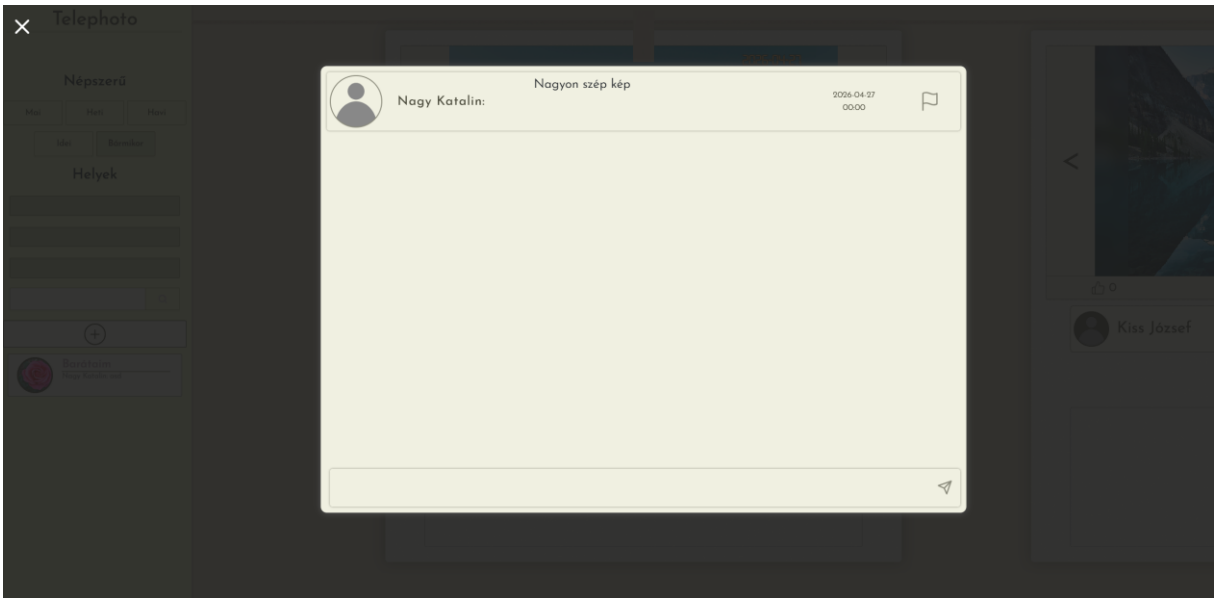
17. kép - Kitöltött feltöltési felület

3.5.3.2 Interaktálás a tartalmakkal

A posztok kártyáiban van lehetőségünk like-kal vagy dislike-kal reagálni a posztokra. Emellett el is menthetjük őket a kedvenceink közé, vagy jelenthetjük őket, ha szabályellenesnek tartjuk őket. Ezeket mind egy-egy piktogrammal jelzett gombbal.

18. kép - Minta poszt

Szintén a posztok kártyáin van lehetőségünk megnyitni a kommenteket, ahol láthatjuk, hogy kik és milyen mondanivalóval reagáltak a posztra, és mi is írhatunk hozzászólást a szabályokat észbentartva. Ha valamelyik komment szövegével problémát észlelünk, jelenthetjük, hasonlóan a posztokhoz.



19. kép - Minta kommentszekció

Amennyiben beszeretnénk zárni a kommentszekciót, azt kétféleképpen tudjuk megtenni, a „háttérre” kattintunk vagy megnyomjuk a bal felső sarokban található „X”-et.

3.5.3.3 Poszt szűrés

A posztokat népszerűség, azon belül idő szerint, véletlenszerű helyszínek, vagy keresett kifejezés alapján tudjuk böngészni. Ezt a népszerűségnél és a helyszíneknél a megfelelő gombokkal tudjuk megtenni, míg a keresésnél elsőnek beírjuk a keresendő kifejezést, majd rákattintunk a mező melletti nagyító ikonra.

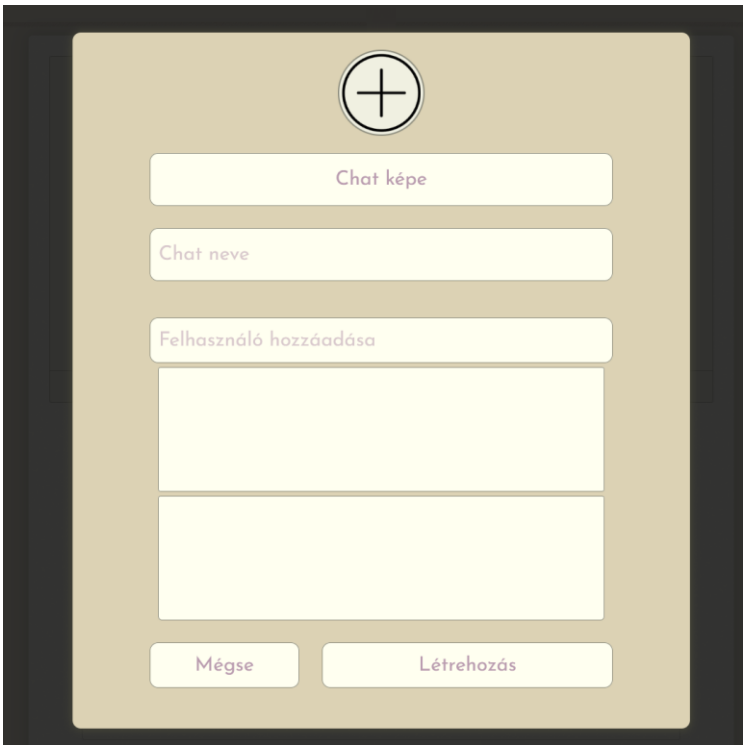
20. kép - Poszt szűrési felület

3.5.3.4 Chat

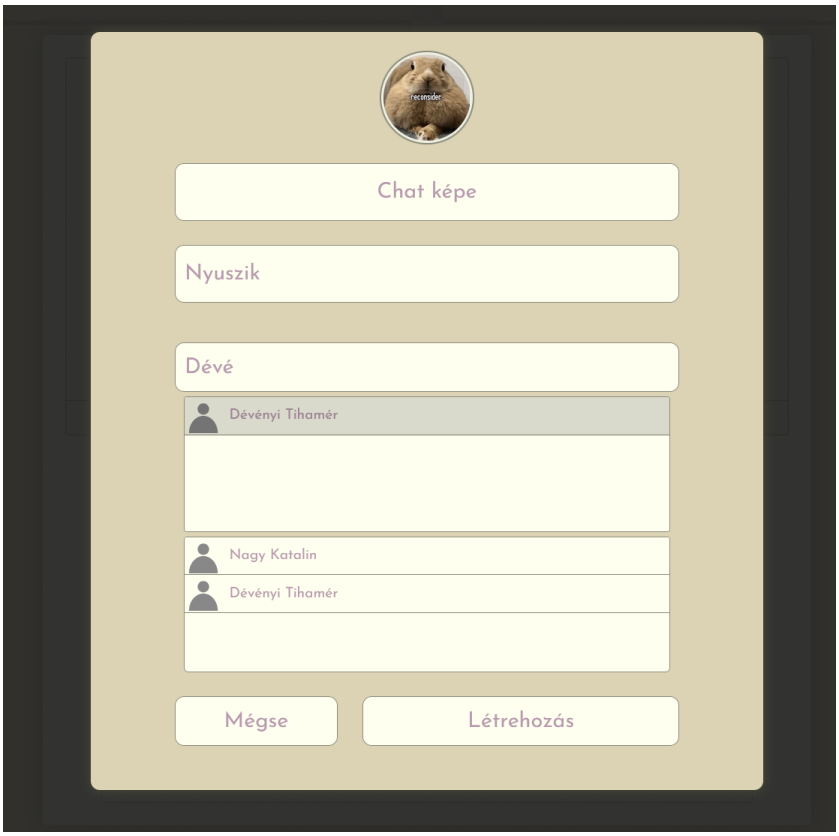
A főoldal bal oldalán elérhető a chat funkció. Itt egy gombbal új beszélgetést hozhatunk létre a felugró ablakban, vagy egy már meglévő beszélgetésre kattintva tudjuk azt folytatni.

21. kép - Chat létrehozása / meglévő chatek

Amennyiben új chatet szeretnénk létrehozni, azt a név nélküli „+” gombra kattintva tudjuk megtenni. A kattintás után megjelenik egy felugró ablak, ahol meg kell adni a beszélgetés képét, nevét, illetve, hogy kiket szeretnénk hozzáadni. A felhasználó hozzáadásához kezdjük el beírni a nevét, majd kattintsunk rá a megjelenő listában. Ha valakit mégse szeretnénk hozzáadni a chathez, abban az esetben kattintsunk a felhasználóra az alsó listában, ezzel eltávolítva az adott személyt.

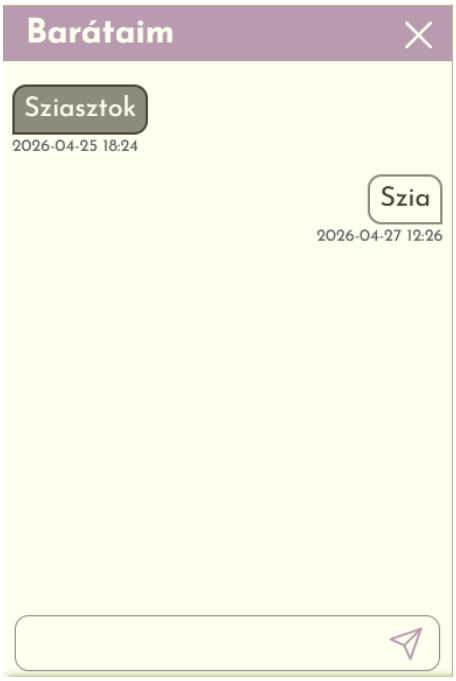


22. kép - Kitöltetlen chat létrehozási felület



23. kép - Kitöltött chat létrehozási felület

A megnyitott chat felületen láthatjuk az üzeneteket, azoknak a dátumát, egy beviteli mezőt ahonnan új üzenetet tudunk küldeni, a chat nevét, illetve egy „X”-et amivel is be tudjuk zárni a beszélgetést.



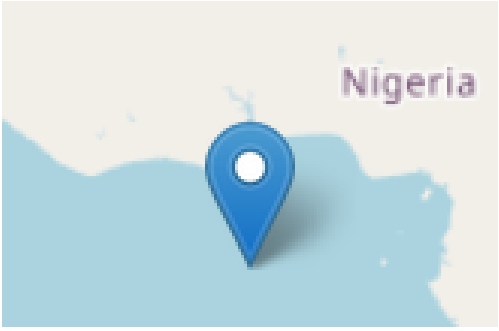
24. kép - Megnyitott chat

3.5.4 Térkép

A navigációs sáv térkép gombjára kattintva átlépünk a térkép nézetre, ahol a posztok a földrajzi koordináta-rendszer adataik alapján vannak megjelölve a világtérképen.

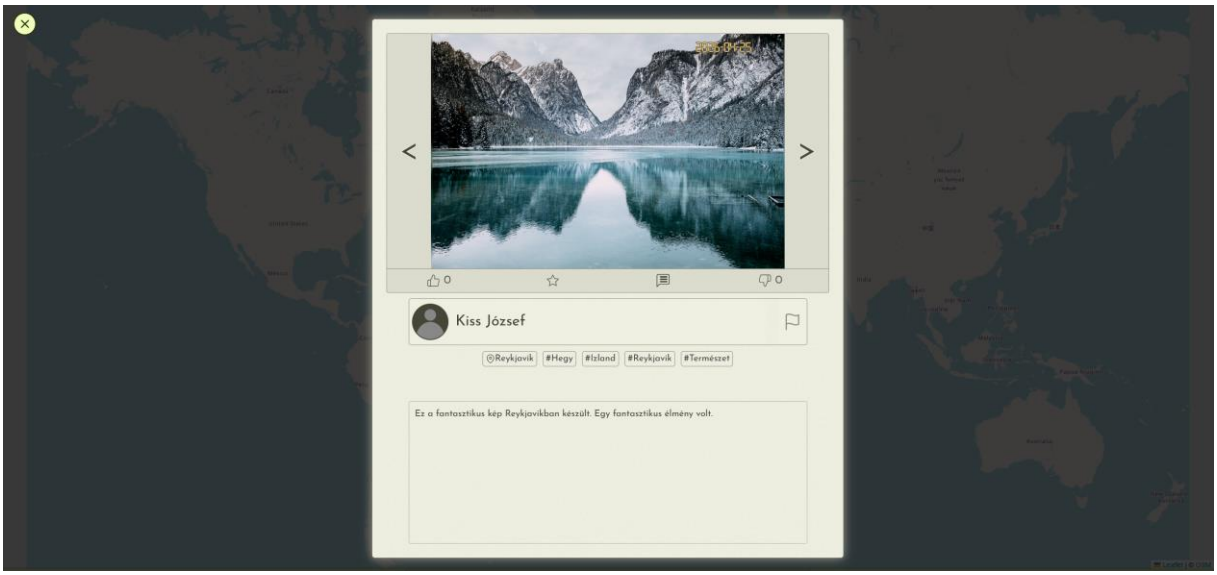
3.5.4.1 Koordináta alapú poszt jelzők

A posztokat meg tudjuk nyitni, amennyiben rákattintunk az azt jelölő jelzőre. A kattintás után megjelenik az előnézete, amennyiben a teljes posztot megszeretnénk megtekinteni, úgy az előnézetre kattintva tudjuk ezt megtenni.



25. kép – Térkép poszt jelző

26. kép - Térkép poszt előnézete

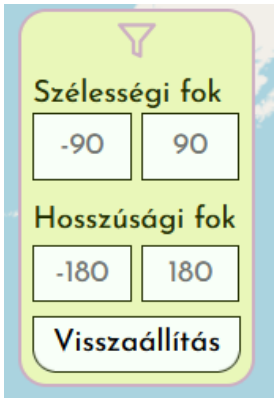


27. kép - Térkép megnyitott poszt

Amennyiben be szeretnénk zárni a posztot, azt a kommentszekcióhoz hasonlóan, a háttérre kattintva, vagy az „X”-el tudjuk megtenni.

3.5.4.2 Szűrés

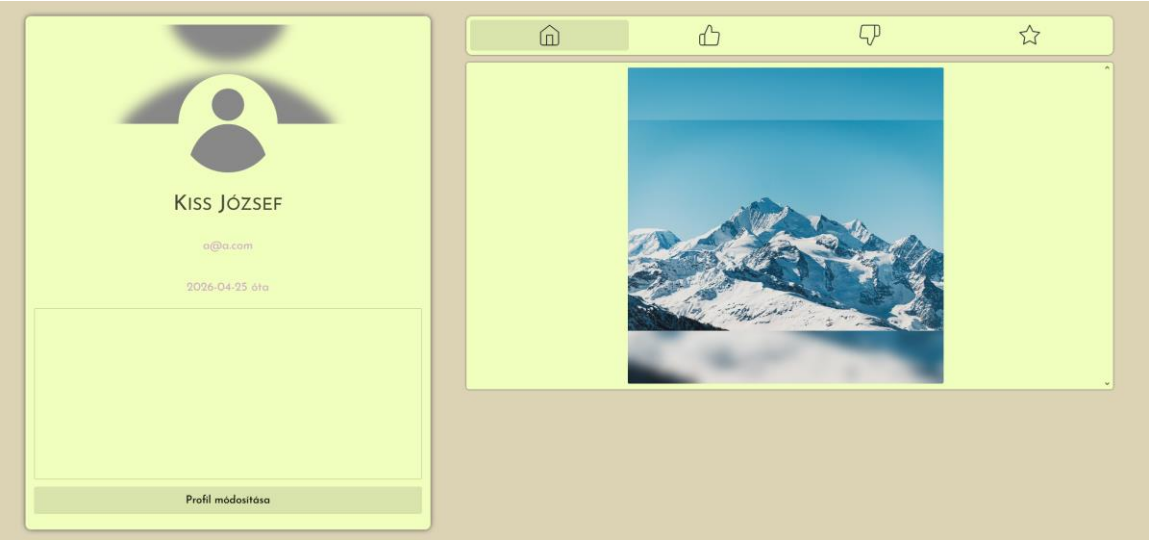
Lehetőségünk van a megjelenő jelzőket hosszúsági és szélességi fok alapján szűrni. Itt megadhatjuk a minimumot és a maximumot, illetve vissza is állíthatjuk azt az eredeti állapotára.



28. kép - Térkép szűrő

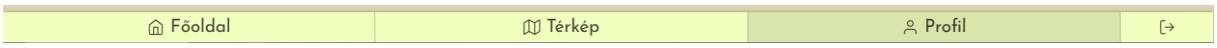
3.5.5 Profil

A navigációs sáv profil gombjára kattintva átlépünk a saját profilunk oldalára. Itt megtekinthetjük a saját adatainkat és posztjainkat, illetve azokat a bejegyzéseket, amiket like-oltunk, dislike-oltunk, vagy elmentettünk.



29. kép - Profil oldal

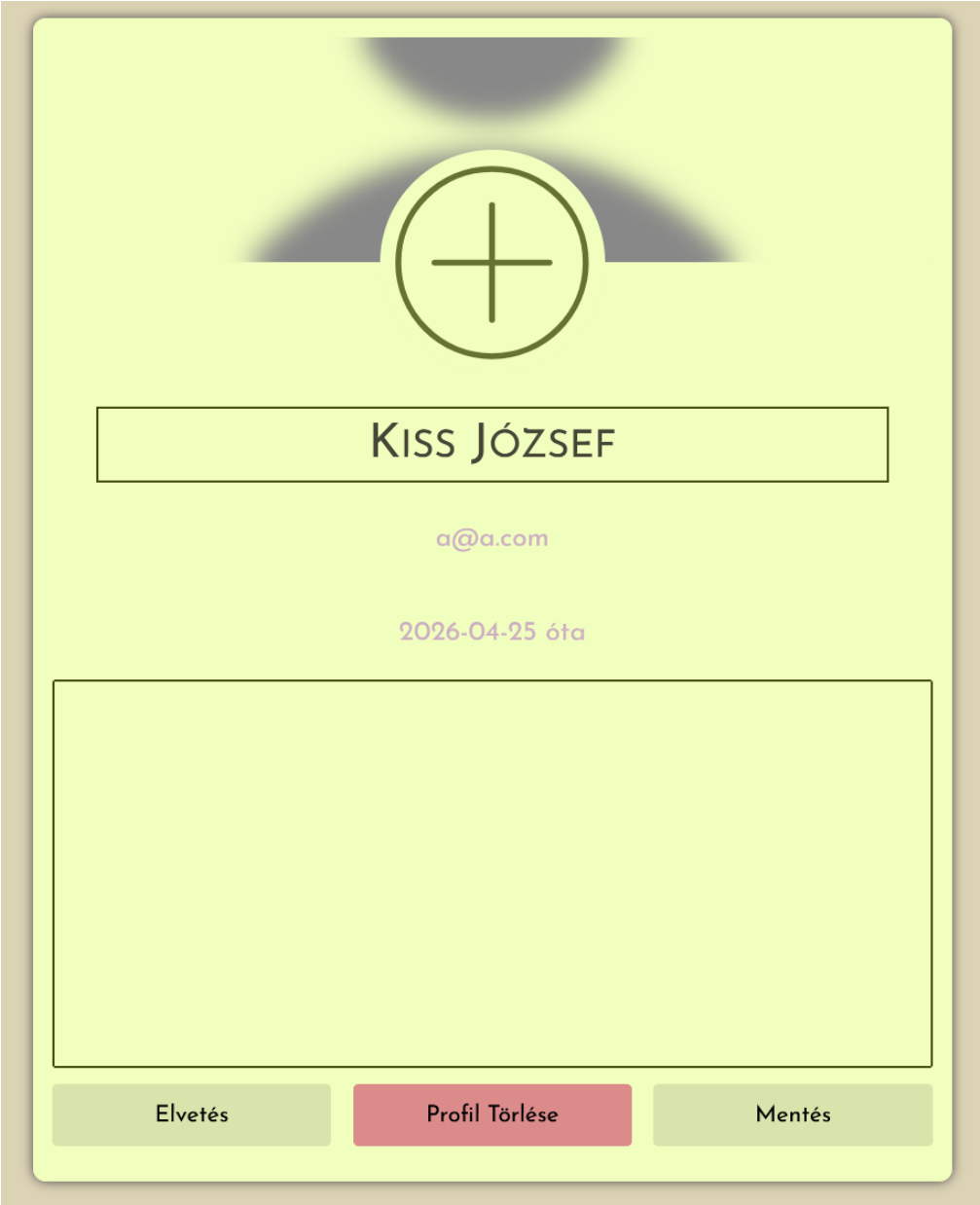
Továbbá a profil oldal sajátossága, hogy innen tudunk kijelentkezni a fiókunkból.



30. kép - Profil oldal navigációs sávja

3.5.5.1 Profil módosítás

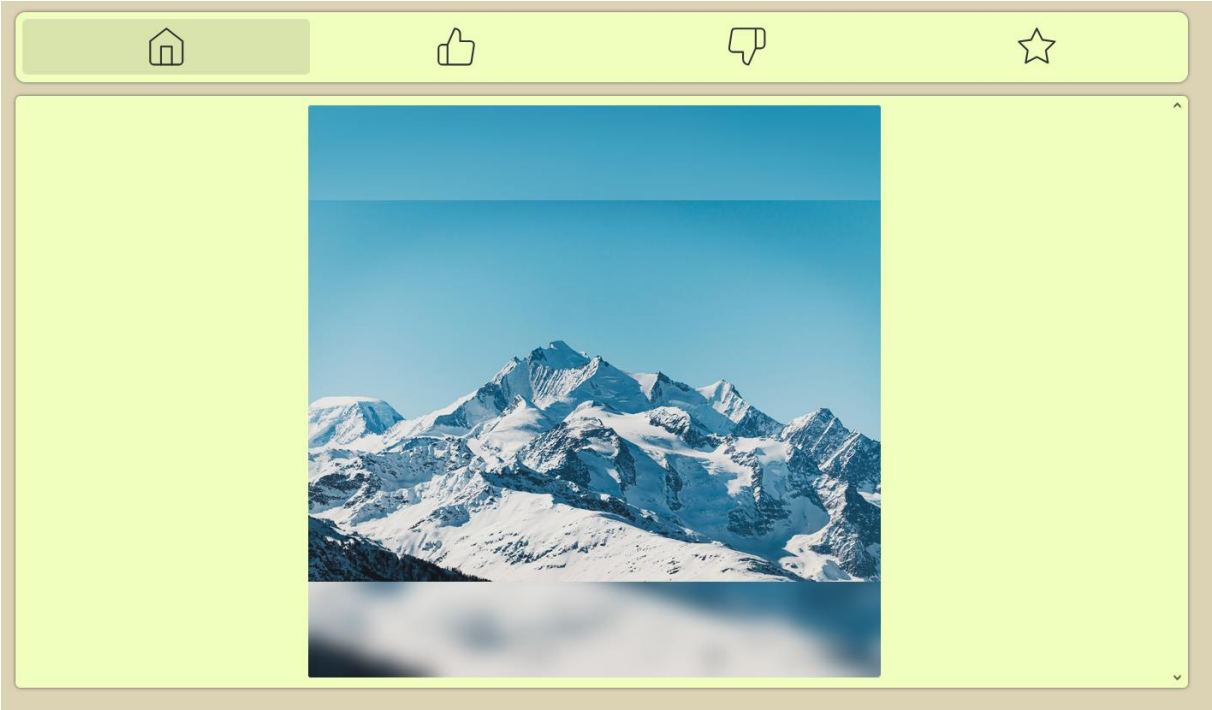
Lehetőségünk van a profilunk bizonyos adatait módosítani, ezek a következőek: név, profilkép, valamint a bemutatkozásunk. Ezt a „profil módosítása” gomb megnyomásával kezdeményezhetjük. Miután ez megtörtént, az oldal kiemeli a megváltoztatható adatokat, illetve három új gomb jön létre, az „elvetés”, ennek segítségével tudjuk visszavonni a változtatásokat, a „profil törlése” gomb, ezzel tudjuk profilunkat véglegesen törölni, illetve a „mentés” gomb, ezzel, amennyiben történt változtatás, azokat el tudjuk menteni.



31. kép - Profil módosítási felület

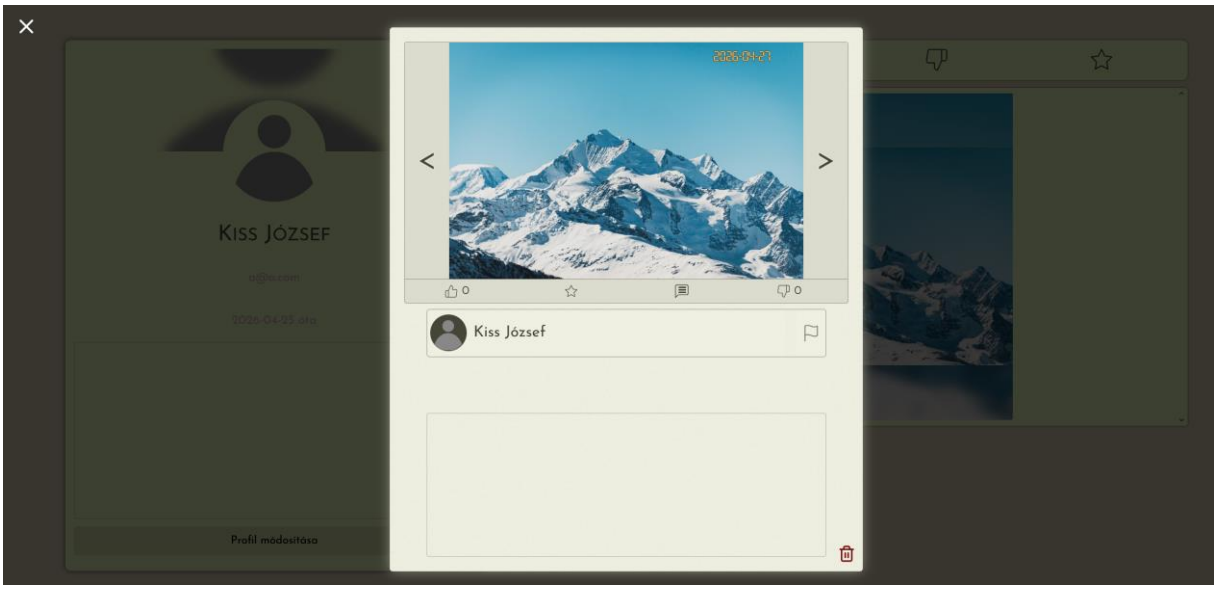
3.5.5.2 Saját / interaktált posztok megtekintése

Lehetőségünk van a feltöltött, illetve korábban már like-olt, dislike-olt vagy elmentett posztok megtekintésére. Ehhez annyi a teendőnk, hogy a felső sávon rákattintunk a megfelelő ikonnal ellátott gombra. A „ház” ikon a saját posztjainkat jelzi, a „felfele mutató hüvelyk ujj” a like-olt, a „lefele mutató hüvelykujj” a dislike-olt, a „csillag” pedig az elmentett posztokat jelenti.



32. kép - Profil a felhasználó posztjai

A betöltött listán meg tudjuk nyitni ezeket a posztokat, így megtekinteni azokat teljes formájukban. Amennyiben a saját posztunkat nyitjuk meg, úgy azt törölni is tudjuk. A poszt jobb alsó sarkában található „kuka” ikon segítségével.



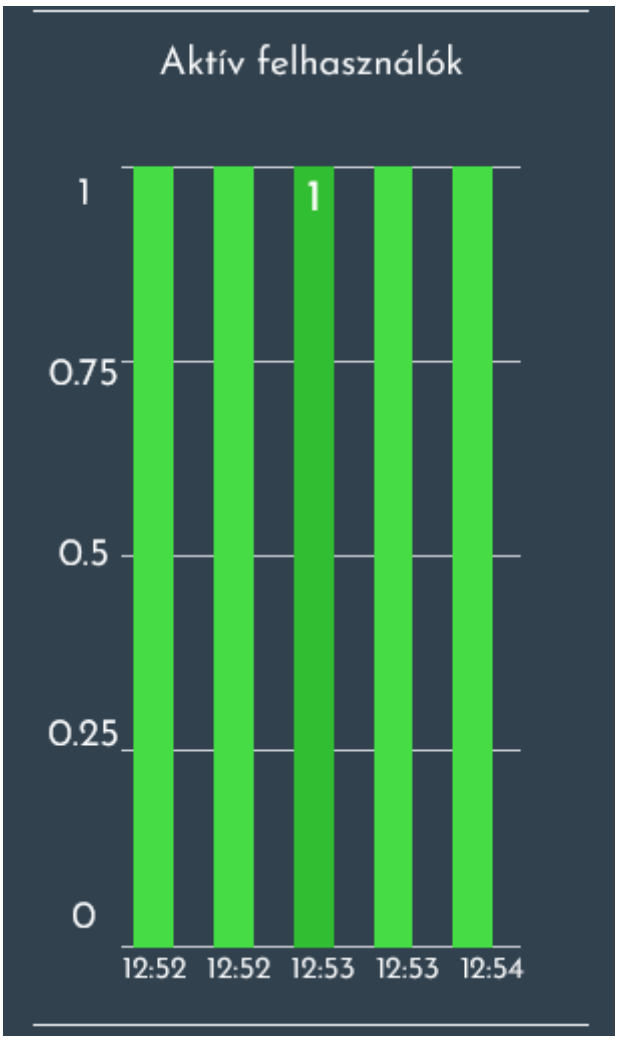
Amennyiben be szeretnénk zárni a posztot, azt a komment szekcióhoz hasonlóan, a háttérre kattintva, vagy az „X”-el tudjuk megtenni.

3.5.6 Admin

Az admin oldalt csak az arra jogosult felhasználó tudja elérni. Amennyiben ehhez a felülethez hozzáférünk, azt a navigációs sávon a „pajzs” ikonnal jelzett gombbal tudjuk elérni. Az admin felület két fő részből áll, a navigációt segítő oldalsó sávból, és a felhasználókat, posztokat és kommenteket tartalmazó fő részből.

3.5.6.1 Aktív felhasználók számának követése

Lehetőségünk van az aktív felhasználók követésére az oldalsó sávon található grafikon segítségével, ami fél percenként frissül. Az oszlop fölé helyezve a kurzort megmutatja, hogy az adott oszlop pontosan mennyi felhasználót jelent.



33. kép - Aktív felhasználók számát követő grafikon

3.5.6.2 Felhasználók kezelése

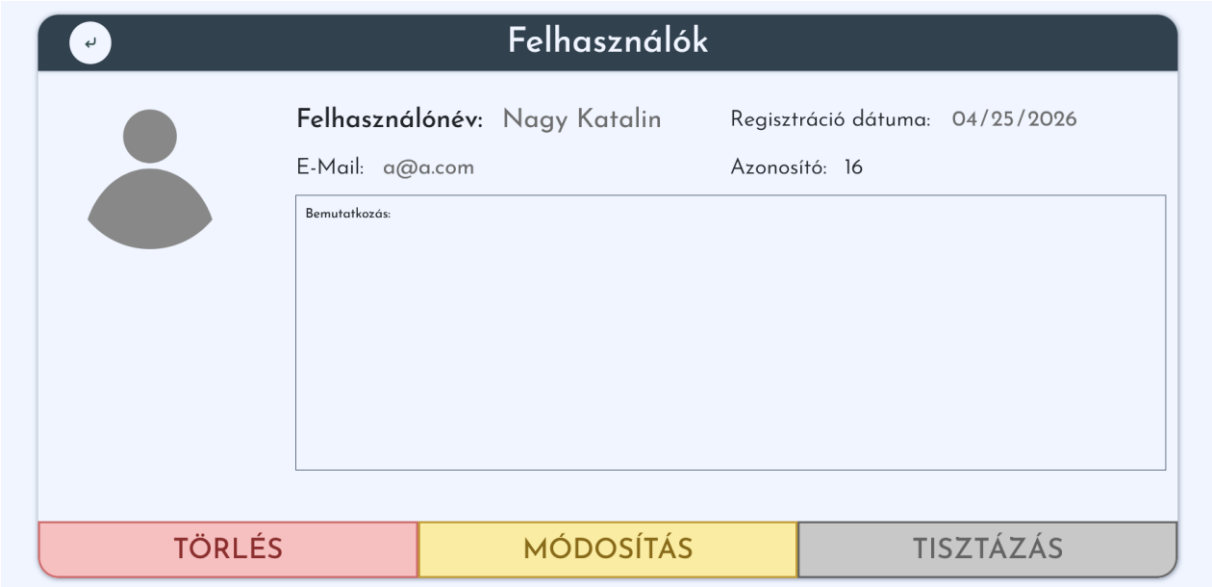
A profilok kezelését a „Felhasználók” című táblában tehetjük meg. Itt egy táblázatban megkapjuk az összes tag azonosítóját, nevét, e-mail címét, regisztrációjának dátumát, posztjainak és kommentjeinek számát. A táblázat ezenfelül jelzi azt is, hogy az adott felhasználó admin-e, ezt kék kiemeléssel, illetve azt is, hogy jelentették-e a felhasználót, ezt piros kiemeléssel.

Felhasználók

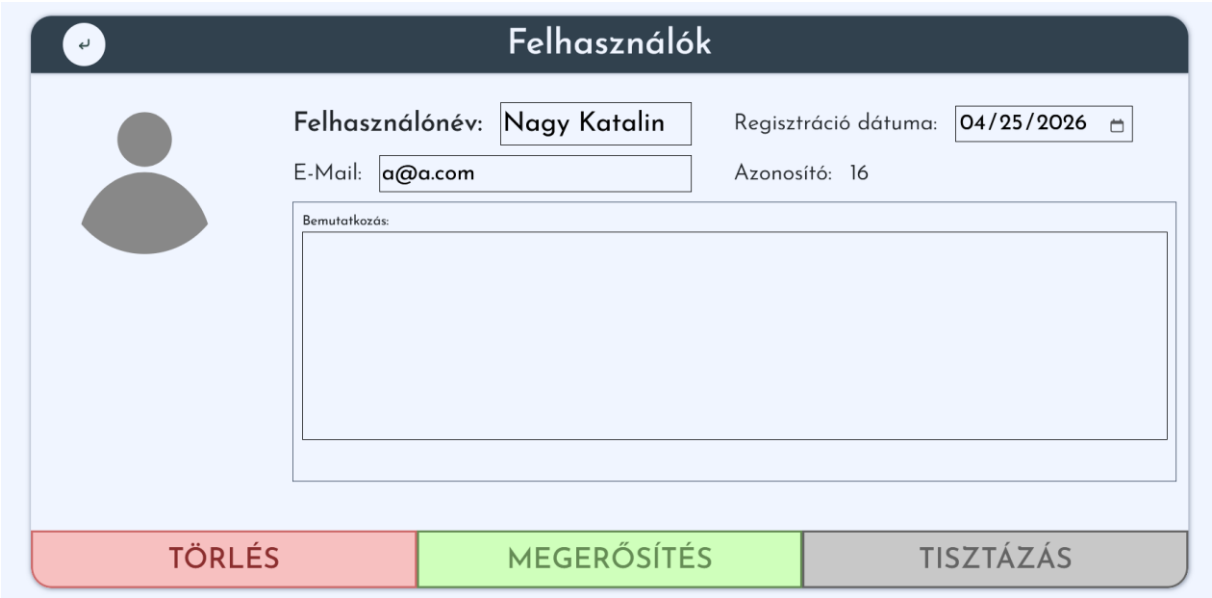
ID	Felhasználónév	E-Mail	Regisztráció Dátuma	#	#
15	Kiss József	a@a.com	2026-04-25 17:51	1	0
16	Nagy Katalin	a@a.com	2026-04-25 00:00	0	0
17	Dévényi Tihamér	a@a.com	2026-04-25 17:52	0	0

34. kép - Felhasználók táblázat

A felhasználóra kattintva megjelenik a felhasználó képe, neve, regisztrációjának dátuma, e-mail címe, azonosítója, bemutatkozása, és a felhasználón végezhető műveletek, ezek a következőek: „törlés”, „módosítás” és „tisztázás”. A törlés gombra kattintva kitudjuk törölni az adott felhasználót, a módosítás gombbal meg tudjuk változtatni a felhasználó adatait, a profilkép és az azonosító kivételével. A változtatások véglegesítéséhez a „módosítás” gomb helyett megjelenő „megerősítés” gombra kell kattintani. A tisztázás gombbal le tudjuk venni a jelentést a felhasználóról (ehhez a felhasználónak elsőnek jelentve kell, hogy legyen). Amennyiben vissza szeretnénk lépni a táblázatra, azt a bal felső sarokban található nyíllal tehetjük meg.



35. kép - Megnyitott felhasználó az admin felületen



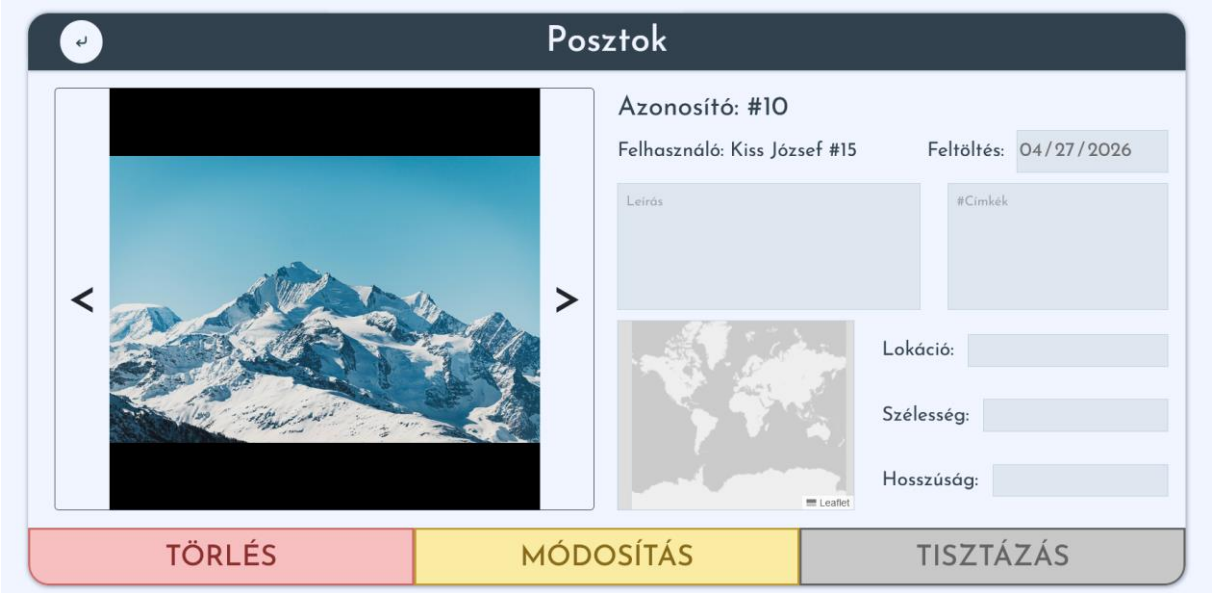
36. kép - Módosítás alatt álló felhasználó az admin felületen

3.5.6.3 Posztok kezelése

A „Felhasználók” táblához hasonlóan működik, láthatjuk egy poszt azonosítóját, feltöltőjét, feltöltésének dátumát, like-ok számát, dislike-ok számát, és a feltöltött tartalmak számát. Ennél a táblánál csak a jelentett posztok vannak jelölve, ugyanúgy piros kiemeléssel.

37. kép - Posztok táblázat

A posztra kattintva megjelenik a(z) tartalma, azonosítója, feltöltőjének neve és azonosítója, feltöltés dátuma, leírása, címkéi, lokációja (amennyiben a szélességi és hosszúsági fokok nem üresek, a poszt jelzője megjelenik a térképen). A felhasználó táblához hasonlóan megjelennek a műveletek, ezeknek a szerepei ugyanazok. Hasonlóan mint korábban, ha vissza szeretnénk térni a táblázatra, azt a bal felső sarokban található nyíl segítségével tehetjük meg.



38. kép - Megnyitott poszt az admin felületen

39. kép - Módosítás alatt álló poszt az admin felületen

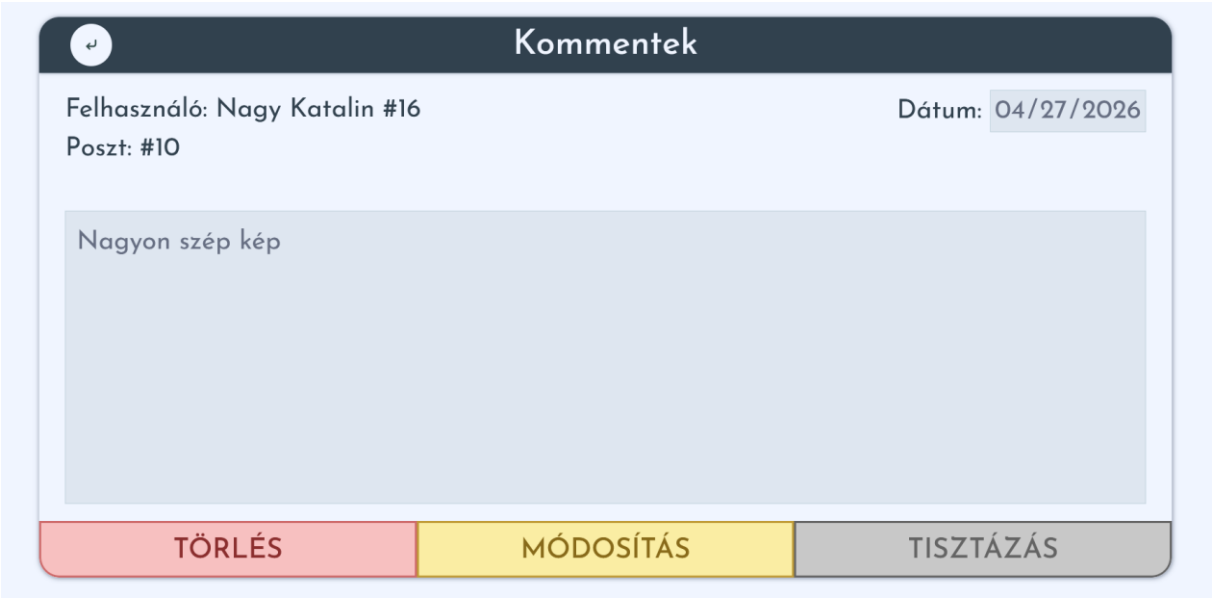
3.5.6.4 Kommentek kezelése

Hasonlóan működik mint a korábbi táblázatok, láthatjuk a komment azonosítóját, az ehhez tartozó poszt azonosítóját, felhasználó nevét, hozzászólás tartalmát és dátumát. A posztok táblázathoz hasonlóan itt is csak a jelentett kommentek vannak kiemelve, szintúgy pirossal.

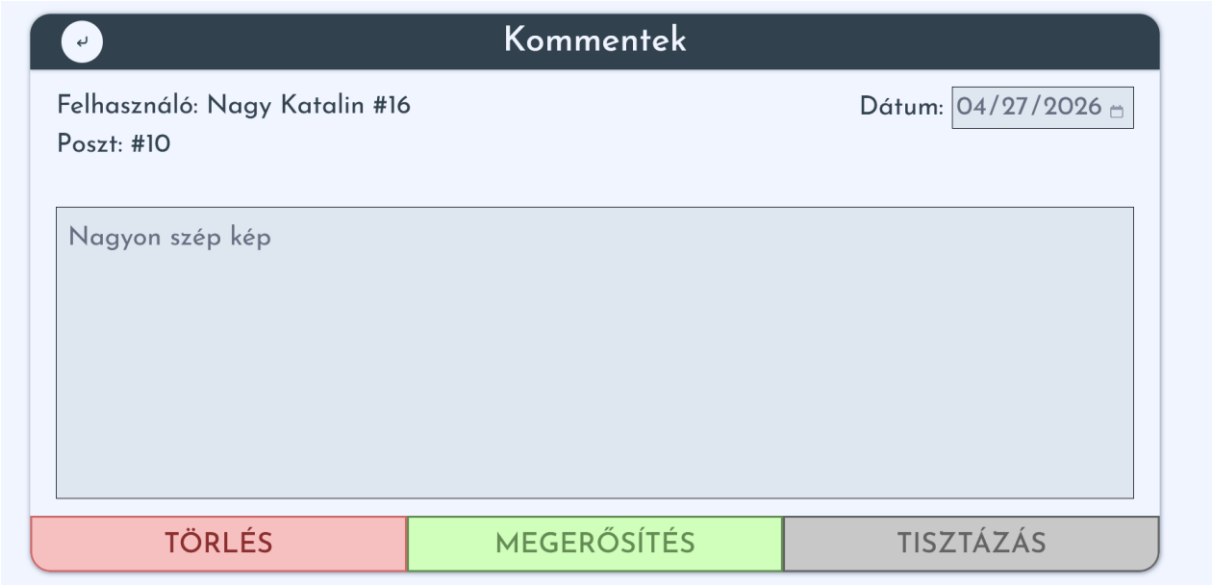
40. kép - Kommentek táblázat

A megnyitott kommentnél láthatjuk a felhasználó nevét és azonosítóját, poszt azonosítóját, hozzászólás dátumát és tartalmát. Ugyanazok a műveletek érhetőek el,

mint az előző részekben. Amennyiben végeztünk, ugyanúgy a bal felső sarokban lévő nyíllal tudunk visszalépni.



41. kép - Megnyitott komment az admin felületen



42. kép - Módosítás alatt álló komment az admin felületen

4 Csapatmunka megvalósítása

A projektet két részre osztottuk, a felhasználó által látott oldalon megjelenő tartalomra és az oldal háttér működésére. Az oldalon megjelenő tartalmakért javarészt felelt: Farkas Tamás. Ez olyan feladatokat takar, mint például a dizájn megalkotása, meglévő elemek újratervezése, alapvető működés elkészítése. Az oldal háttér működéséért feleltek: Barnai Benjámin, Hegedűs Kevin. Ez olyan feladatokat takar, mint például az adatbázis megtervezése, végpontok és komplex függvények elkészítése.

A verziókezeléshez githubot, míg a kommunikációhoz discordot használtunk. A külön feladatokhoz külön brancheket hoztunk létre, és amikor mindenki kész lett az adott feladatával összeültünk és együtt mergeltük az elkészült brancheket.

Habár megbeszéltük, hogy ki miért felelős, ezek a határok nem voltak kőbe vésve, ezzel esetlegesen tudtuk egymást elősegíteni, amennyiben valaki megakadt volna.

Összeségében szerintünk sikerült felosztani a projektet mindenkinek megfelelő módon.

5 Összefoglalás

A beharangozott funkciók mindegyikét sikerült implementálni a kész projektben. A projekt készítse közben eszünkbe ötlöttek új funkciók és potenciális fejlesztési lehetőségek.

A projekt során rengeteget tanultunk a dizájnolásról, hogy hogyan is kell egy intuitív és felhasználó barát oldalt készíteni. Továbbá megtanultunk használni különféle keretrendszereket. Volt sok újratervezés és vita az oldal kinézetéről, de végül sikerült egy mindenki által elfogadott verziót kialakítani. A legnagyobb kihívást az állította elénk, hogy a kisebb megjelenő hibákat identifikáljuk, értelmezzük és végső soron megoldjuk azokat.

Most már látjuk, hogy mit lehetett volna szebben és vagy gyorsabban készíteni. Rájöttünk, hogy milyen fontos is utána nézni az ismeretlen témáknak, mielőtt vakon nekiállnánk kódolni. Felfedeztünk rengeteg új funkciót, amikről eddig nem is hallottunk.

Megmutatta ez a projekt, hogy valójában mennyire nehéz is egy ilyen közösségi média platformot létrehozni. Megtaláltuk a tudásunk határait és megpróbáltuk feszegetni azokat.

Összességében büszkék vagyunk magunkra, hogy egy számunkra komplex projektet sikerült megalkotnunk.

6 Ábrajegyzék

6.1 Képek

1. kép - Az adatbázis modellje 5
2. kép - HashString függvény kódrészlete 11
3. kép - VerifyHashedString függvény kódrészlete 12
4. kép - A postTypeOffset objektum felépítése, és a /setOffset végpont kódrészlete 12
5. kép - topPosts függvény kódrészlete 13
6. kép - A generateBarChart kódrészlete 14
7. kép - A registration kódrészlete 1/2 15
8. kép - A registration kódrészlete 2/2 15
9. kép - Az uploadPost kódrészlete 16
10. kép - A hangPictures kódrészlete 1/2 17
11. kép - A hangPictures kódrészlete 2/2 18
12. kép - A getChatData kódrészlete 19
13. kép - A filterPins kódrészlete 20
14. kép - Bejelentkezési felület 36
15. kép - Regisztrációs felület 37
16. kép - Kitöltetlen feltöltési felület 38
17. kép - Kitöltött feltöltési felület 38
18. kép - Minta poszt 39
19. kép - Minta kommentszekció 40
20. kép - Poszt szűrési felület 40
21. kép - Chat létrehozása / meglévő chatek 41
22. kép - Kitöltetlen chat létrehozási felület 41
23. kép - Kitöltött chat létrehozási felület 42
24. kép - Megnyitott chat 42
25. kép – Térkép poszt jelző 43
26. kép - Térkép poszt előnézete 43
27. kép - Térkép megnyitott poszt 44
28. kép - Térkép szűrő 44
29. kép - Profil oldal 45
30. kép - Profil oldal navigációs sávja 45

31. kép - Profil módosítási felület 46
32. kép - Profil a felhasználó posztjai 47
33. kép - Aktív felhasználók számát követő grafikon 48
34. kép - Felhasználók táblázat 49
35. kép - Megnyitott felhasználó az admin felületen 50
36. kép - Módosítás alatt álló felhasználó az admin felületen 50
37. kép - Posztok táblázat 51
38. kép - Megnyitott poszt az admin felületen 51
39. kép - Módosítás alatt álló poszt az admin felületen 52
40. kép - Kommentek táblázat 52
41. kép - Megnyitott komment az admin felületen 53
42. kép - Módosítás alatt álló komment az admin felületen 53

6.2 Táblázatok

1. táblázat - Az adatbázis kapcsolatainak táblázata 6
2. táblázat - A chats táblának a mezői 6
3. táblázat - A users táblának a mezői 7
4. táblázat - A posts táblának a mezői 7
5. táblázat - A pictures táblának a mezői 8
6. táblázat - A interactions táblának a mezői 8
7. táblázat - A favorites táblának a mezői 8
8. táblázat - A comments táblának a mezői 9
9. táblázat - A chat_members táblának a mezői 9
10. táblázat - A messages táblának a mezői 10
11. táblázat - A util.js tesztesetei 25
12. táblázat - Az admin.js tesztesetei 25
13. táblázat - A registration.js tesztesetei 27
14. táblázat - A login.js tesztesetei 27
15. táblázat - A chat.js tesztesetei 28
16. táblázat - Az uploadPost.js tesztesetei 29
17. táblázat - A profile.js tesztesetei 30
18. táblázat - A map.js tesztesetei 30

19. táblázat - A post.js tesztesetei 31
20. táblázat - Teszt lefedettség..... 31

7 Irodalomjegyzék

(2025. december 20). Forrás: NodeJS: https://nodejs.org/api/crypto.html

(2025. december 20). Forrás: W3schools: https://www.w3schools.com/nodejs/nodejs_crypto.asp

(2026. április 6). Forrás: Stackoverflow: https://stackoverflow.com/questions/990904/remove-accents-diacritics-in-a-string-in-javascript

(2026. január 26). Forrás: Stackoverflow: https://stackoverflow.com/questions/3922739/limit-text-length-to-n-lines-using-css

(2026. január 26). Forrás: SVG repo: https://www.svgrepo.com/svg/423404/plus

(2026. április 6). Forrás: Stackoverflow: https://stackoverflow.com/questions/63880776/how-to-display-an-image-which-i-inputted

(2026. április 6). Forrás: MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/API/URL

(2026. április 6). Forrás: Socket.IO: https://socket.io/docs/v4/

(2026. április 6). Forrás: Socket.IO: https://socket.io/docs/v4/client-socket-instance/

(2026. április 6). Forrás: Socket.IO: https://socket.io/docs/v4/rooms/

(2026. április 6). Forrás: Socket.IO: https://socket.io/docs/v4/tutorial/api-overview

(2026. április 12). Forrás: W3schools: https://www.w3schools.com/tags/tag_video.asp

(2026. április 12). Forrás: Stackoverflow: https://stackoverflow.com/questions/7682655/video-with-custom-controls-using-html5-video-and-javascript

(2026. április 12). Forrás: MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/API/HTMLMediaElement/currentTime

(2026. április 12). Forrás: MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/API/HTMLMediaElement/duration

(2026. április 12). Forrás: Stackoverflow: https://stackoverflow.com/questions/5958132/javascript-to-stop-html5-video-playback-on-modal-window-close

(2026. március 24). Forrás: Stackoverflow: https://stackoverflow.com/questions/18481308/set-mouse-wheel-to-horizontal-scroll-using-css

(2026. március 24). Forrás: Geeks for geeks: https://www.geeksforgeeks.org/javascript/introduction-to-intersection-observer/

(2026. március 24). Forrás: MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/API/IntersectionObserver

(2026. március 24). Forrás: MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API

(2026. április 6). Forrás: NPM: https://www.npmjs.com/package/multer

(2026. február 24). Forrás: Github: https://github.com/Leaflet/Leaflet.markercluster?tab=readme-ov-file#customising-the-clustered-markers

(2026. január 6). Forrás: OpenFreeMap: https://openfreemap.org/quick_start/

(2026. április 19). Forrás: Jest: https://jestjs.io/docs/getting-started

(2026. április 19). Forrás: Jest: https://jestjs.io/docs/expect

(2026. április 19). Forrás: Jest: https://jestjs.io/docs/mock-function-api

(2026. április 6). Forrás: W3schools: https://www.w3schools.com/jsref/met_win_matchmedia.asp

(2026. április 6). Forrás: MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/API/Window/matchMedia

(2026. április 12). Forrás: Feather icons: https://feathericons.com/

(2026. április 24). Forrás: Claude AI: https://claude.ai/

(2026. április 9). Forrás: MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Scrollbars_styling

(2026.. április 24). Forrás: SQLFlow: https://sqlflow.gudusoft.com/#/

Agafonkin, V. (2026. április 6). Forrás: Leaflet: https://leafletjs.com/reference.html

Sidorov, A. (2026. január 31). Forrás: MySQL2: https://sidorares.github.io/node-mysql2/docs

Wallander, M. (2026. január 17). Forrás: Github: https://github.com/mattiasw/ExifReader